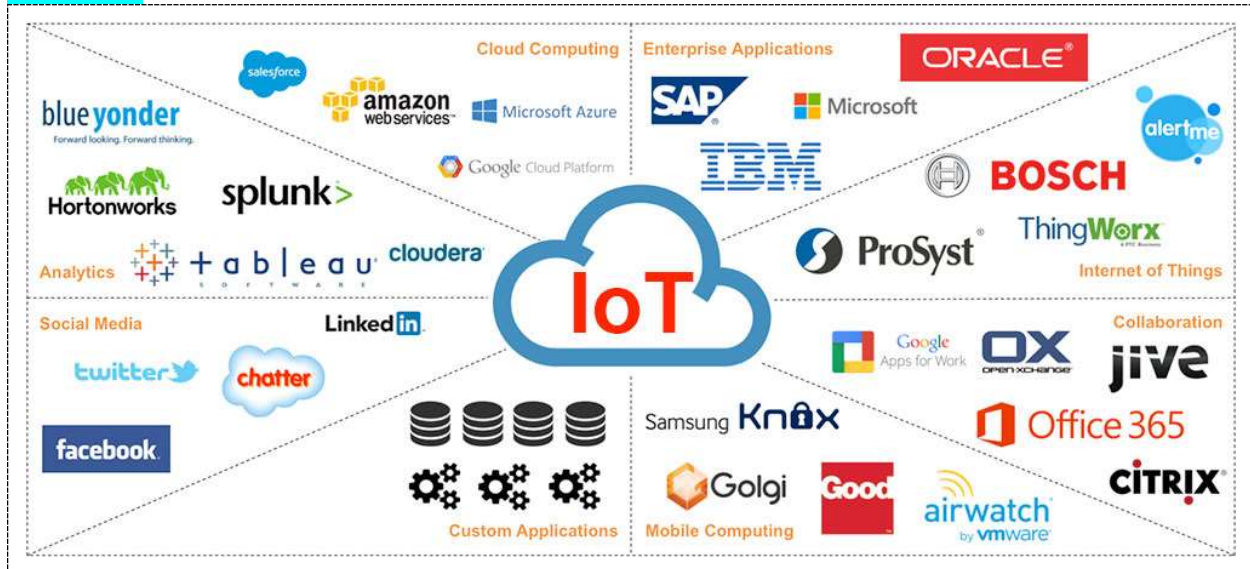


1/3: MQTT Protocol (Update 20260315)

<https://techtutorialsx.com/2017/04/24/esp32-publishing-messages-to-mqtt-topic/>

<https://www.ioxhop.com/article/74/esp32-เบื้องต้น-บทที่-13-เชื่อมต่อกับ-mqtt>

1a. เนื้อหา

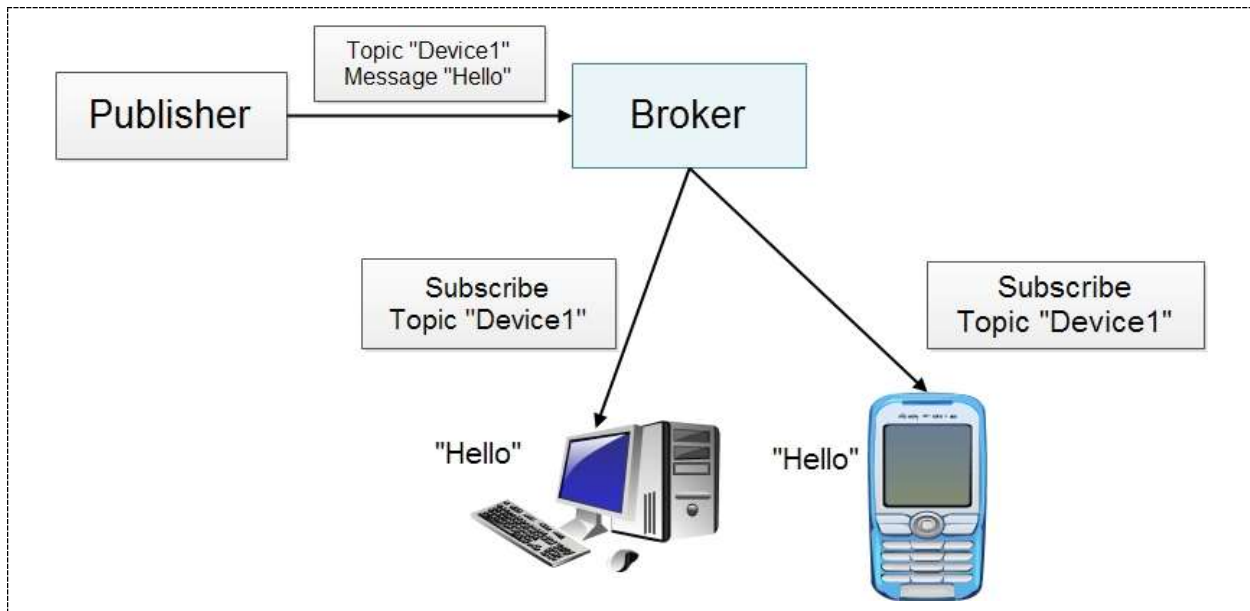


1.1 IoT Concept

ปัจจุบันเทคโนโลยีที่กำลังมาแรงสำหรับนักพัฒนาด้าน Embedded (ไม่ได้แค่เฉพาะ Embedded อย่างเดียวครับ) เป็นเทคโนโลยีที่กล่าวกันมากคงจะหนีไม่พ้น IoT ซึ่งเป็นเทคโนโลยีใหม่ในยุคนี้เลยก็ว่าได้ แรงขนาดที่ว่า Microsoft เอง ก็ยังพอร์ต Windows 10 มาวิ่งเล่นบน Raspberry Pi แถมยังใจดีติด IoT มาให้ด้วย ซึ่งผมยังไม่ได้ตามลงไปดูว่าใช้ Broker ตัวไหน และมี Library ให้ใช้งานมาด้วยหรือไม่? หรือไม่ผมก็เข้าใจผิดเกี่ยวกับมันครับถ้าผิดพลาดก็ขออภัยมานะที่นี้ด้วยครับ.

IoT มันคืออะไร พอลันดูมีหลายสิ่งคืออธิบายไว้มากมาย เช่น [Internet of Things เมื่อคอมพิวเตอร์เริ่มคุยกันเองได้](#), [โลกแห่ง IoT มาถึงแล้ว](#), [IoT เทคโนโลยีที่ธุรกิจต้องรู้](#). ลองนึกภาพดูครับว่าถ้าหากอุปกรณ์สามารถส่งงานไปมาหากันได้ผ่าน www ไม่ว่าจะเป็น PC, Smart Phone หรือแม้แต่อุปกรณ์ขนาดเล็กพวก Micro-Controller, PLC, HUB, Switch หรืออะไรก็แล้วแต่ที่มันสามารถต่อระบบ Network ได้ว่ามันจะอยู่ที่บ้าน ที่โรงงาน ไร่ นา ฟาร์ม โรงเรือน โรงงานอุตสาหกรรมหรือที่อื่นๆที่มีระบบเน็ตเวิร์กที่เข้าถึง www ได้เราจะสามารถควบคุมมันได้ทั้งหมดที่ไหนก็ได้ในโลกใบนี้

IoT ทำมีวิธีการทำงานอย่างไร องค์ประกอบหลักของ IoT จะมี 3 ส่วนคือ Broker, Publisher และ Subscriber. ซึ่งการรับและส่งข้อมูลนั้นมันจะส่งข้อมูลไปมาหากันนั้นจะส่งผ่านตัวกลางนั่นก็คือ Broker Server โดยตัวส่งนี้จะเรียกว่า Publisher ส่งข้อมูลขึ้นไปยัง Broker พร้อมระบุหัวข้อ (Topic) ที่ต้องการส่งข้อมูลออกไป จากนั้นตัวรับซึ่งเรียกว่า Subscriber ถ้าหากตัวรับต้องการรับข้อมูลจากตัวส่งจะต้องทำการ Subscribe หัวข้อ Topic ของ Publisher นั้นๆ ผ่าน Broker เช่นกัน



จากรูปภาพด้านบนจะมีตัว Publisher ทำการ Publish ข้อความ “Hello” ใน Topic Device1 เมื่อและถ้าหากมีคอมพิวเตอร์ หรืออุปกรณ์อื่นๆทำการ Subscribe หัวข้อ Topic Device1 เมื่อ Publisher ทำการส่งข้อมูลไปยัง Topic อุปกรณ์ Subscribe จะได้ข้อความ “Hello” เช่นเดียวกัน. ก็คล้ายๆกับที่ใช้งานไลน์ที่คุยกันเป็นกลุ่มนั้นเลยครับ. ซึ่งจะเห็นข้อความ “Hello” ในเวลาเดียวกันนั้นหมายความว่าอุปกรณ์ใดๆที่ทำการ Subscribe Topic เดียวกันก็จะได้รับข้อความเดียวกันครับ

1.2 MQTT-Message Queue Telemetry Transport

โปรโตคอลที่ใช้สำหรับรับและส่งข้อมูลนั้นคือ MQTT ปัจจุบันถึง Version 3.1 ในที่นี้จะมาทำการทดลองส่งข้อมูลกันตัว Server จะมีอยู่ด้วยกันหลายค่ายครับสำหรับที่ลิสมาด้านล่างนี้ครับ

<u>Open Source MQTT Broker Server</u>	<u>Client</u>
• Mosquitto • RSMB • ActiveMQ • Apollo • Moquette • Mosca • RabbitMQ	• Paho • Xenqtt • mqtt.js • node_mqtt_client • Ascoltatori • Arduino MQTT Client

สำหรับ MQTT Broker Server ฟรีที่ใช้ทดสอบ ดังนี้

- test.mosquitto.org
- mqtt.eclipse.org
- broker.mqttdashboard.com

1.3 Free Public MQTT Brokers (Private & Public)

Posted by Maulin Tolia | Jan 30, 2019

<https://mntolia.com/10-free-public-private-mqtt-brokers-for-testing-prototyping/>

I have listed a few free public and private MQTT brokers that you can use for testing or prototyping.

Private MQTT Broker: In a private broker, only devices that you set can publish and subscribe to the topics on the broker. You should use this for production & prototyping.

Public MQTT Broker: In a public broker, any device can publish and subscribe to topics on the broker. There is no privacy. Public brokers should never be used in production. If you simply want to learn or play around with MQTT then use this.

If you want to install your own broker on your computer or virtual machine see this:
Installing Mosquitto Broker on Ubuntu/Windows/Debian

The information in the tables were last updated on April 7th 2020.

Public MQTT Brokers

Name	Broker Address	TCP Port	TLS Port	WebSocket Port	Message Retention	Persistent Session
Eclipse	mqtt.eclipse.org	1883	N/A	80, 443	YES	YES
Mosquitto	test.mosquitto.org	1883	8883, 8884	80	YES	YES
HiveMQ	broker.hivemq.com	1883	N/A	8000	YES	YES
Flespi	mqtt.flespi.io	1883	8883	80, 443	YES	YES
Dioty	mqtt.dioty.co	1883	8883	8080, 8880	YES	YES
Fluux	mqtt.fluux.io	1883	8883	N/A	N/A	N/A
EMQX	broker.emqx.io	1883	8883	8083	YES	YES
Eclipse	mqtt.eclipse.org	1883	N/A	80, 443	YES	YES

Private MQTT Brokers

Name	Broker Address	TCP Port	TLS Port	WebSocket Port	Message Retention	Persistent Session
Azure	NO	8883	443	NO	Limited	0, 1
AWS	NO	8883	443	NO	Limited	0, 1
CloudMQTT	Custom Port	Custom Port	Custom Port	NOT SURE	YES	0, 1, 2

1.4 IOT และ MQTT คือ อะไร?

MQTT ย่อมาจาก Message Queue Telemetry Transport เป็นโปรโตคอลประยุกต์ที่ใช้โปรโตคอล TCP เป็นรากฐาน ออกแบบมาสำหรับงานที่ต้องการ ๑ สื่อสารแบบเรียลไทม์แบบไม่จำกัดแพลตฟอร์ม หมายถึงอุปกรณ์ทุกชิ้นสามารถสื่อสารกันผ่าน MQTT

MQTT จะแบ่งเป็น 2 ฝ่าย คือฝั่งเซิร์ฟเวอร์มักจะเรียกว่า MQTT Broker ส่วนฝั่งผู้ใช้งานจะเรียกว่า MQTT Client ในการใช้งานด้าน IoT จะเกี่ยวข้องกับ MQTT Client เป็นหลัก โดยจะมี MQTT Broker ทั้งแบบฟรี และเสียเงินไว้รองรับอยู่แล้ว ทำให้การสื่อสารข้อมูลผ่าน MQTT จะใช้เซิร์ฟเวอร์ฟรี หรือ MQTT Broker ฟรี เหล่านั้นเป็นตัวกลาง ลักษณะการใช้งาน MQTT อาจจะเปรียบเสมือนได้กับการใช้งานห้องแชท Line สำหรับอุปกรณ์ โดยอุปกรณ์แต่ละตัวจะมีชื่อเป็นของตนเอง มี Username Password เป็นของตัวเอง และอาจจะมีห้องลับเฉพาะของตนเอง ดังนั้นการใช้งาน MQTT ผู้เขียนจึงจะขอยกตัวอย่างของ MQTT เทียบกับห้องแชทได้ดังนี้

กลุ่มผู้ใช้ (User)

ใน MQTT จะแบ่งกลุ่มของผู้ใช้งานออกเป็น 2 ระดับ คือ

- ระดับสูงสุด – สามารถที่จะรับ-ส่งข้อมูลกับอุปกรณ์ หรือช่องทางใด ๆ ก็ได้ในระบบ หรือเปรียบได้กับแอดมินที่สามารถเข้าไปดูข้อความได้ทุกห้องแม้จะเป็นห้องลับก็ตาม
- ระดับทั่วไป – สามารถรับ-ส่งข้อมูลกับอุปกรณ์หรือช่องทางที่กำหนดไว้เฉพาะเท่านั้น เปรียบได้กับผู้ใช้ Line ที่สามารถแชทในห้องที่ตัวเองสร้างได้ หรือเป็นสมาชิกในห้อง แต่ไม่สามารถเข้าไปแชทในห้องที่ไม่ได้เป็นสมาชิก

ในการใช้งานจริง ในอุปกรณ์ต่าง ๆ ควรจะใช้งานในระดับทั่วไป เพื่อความปลอดภัยกรณีอุปกรณ์เหล่านั้นถูกแฮกแล้วไม่สร้างความเสียหายไปยังอุปกรณ์อื่น ๆ ที่อยู่ในช่องทางเฉพาะของแต่ละอุปกรณ์

เส้นทาง (Topic)

เส้นทาง เปรียบเหมือนกับหัวข้อ หรือห้องแชทที่ต้องการจะคุย และการคุยกันจะมีเฉพาะอุปกรณ์ที่อยู่ในห้องนั้น ๆ (Subscribe) ถึงจะสามารถได้รับข้อมูลที่มีการส่งไปในห้องนั้น ๆ ที่ถูกเรียกว่าเส้นทางเนื่องจากการใช้งานส่งข้อมูลและรับข้อมูลจะเหมือนกับเส้นทางในระบบไฟล์ เช่น /Room1/LED ซึ่งระบบเส้นทางนี้นอกจากอุปกรณ์จะสามารถรอการสนทนาในห้องตามเส้นทาง /Room1/LED ได้แล้ว ยังสามารถรอสนทนาเส้นทาง /Room1 ได้ด้วย หากเป็นการรอฟังในเส้นทาง (Subscribe) /Room1 จะหมายถึงการส่งข้อมูลใด ๆ ที่นำหน้าด้วย /Room1 เช่น /Room1/LED , /Room1/Value ผู้ที่รอฟัง (Subscribe) /Room1 อยู่จะได้รับข้อมูลเหล่านั้นด้วย

คุณภาพข้อมูล (QoS)

แบ่งออกเป็น 3 ระดับดังนี้

- QoS0 – ส่งข้อมูลเพียงครั้งเดียว ไม่สนใจว่าผู้รับจะได้รับหรือไม่
- QoS1 – ส่งข้อมูลเพียงครั้งเดียว ไม่สนใจว่าผู้รับจะได้รับหรือไม่ แต่ให้จำค่าที่ส่งล่าสุดไว้ เมื่อมีการเชื่อมต่อใหม่จะได้รับข้อมูลครั้งล่าสุดอีกครั้ง
- QoS2 – ส่งข้อมูลหลาย ๆ ครั้งจนกว่าปลายทางจะได้รับข้อมูล มีข้อเสียที่สามารถทำงานได้ช้ากว่า QoS0 และ QoS1

การส่งข้อมูล (Publish)

การส่งข้อมูลในแต่ละครั้งจะต้องประกอบไปด้วยเส้นทาง (Topic) ข้อมูล และคุณภาพข้อมูล ซึ่งการส่งข้อมูลจะเรียกว่า Publish

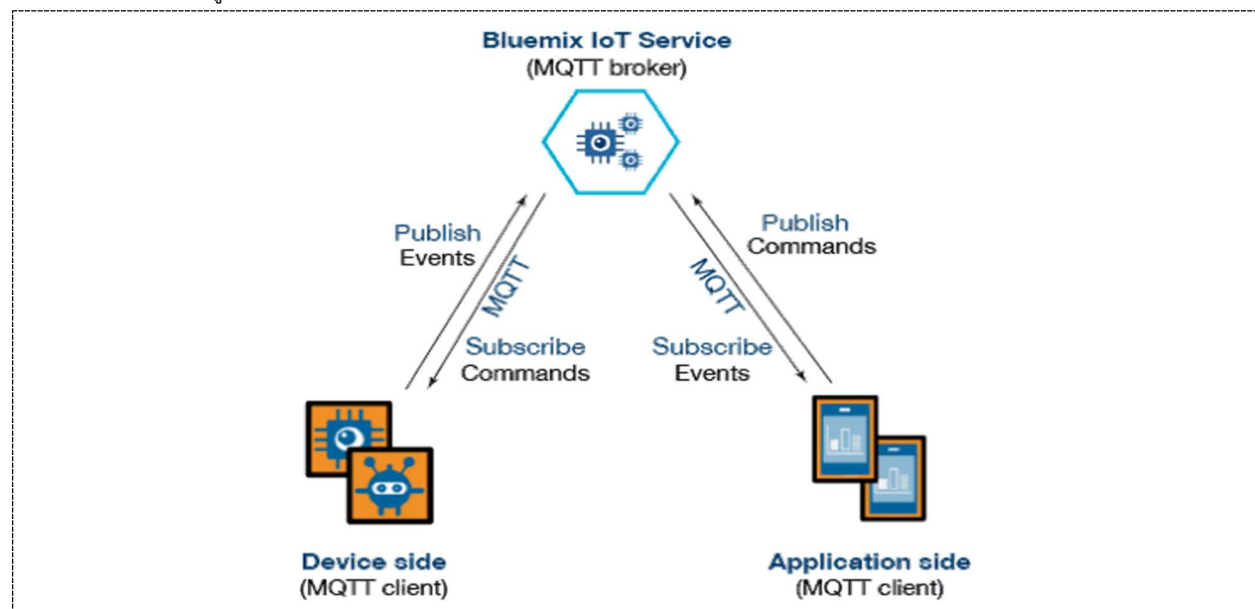
การรับข้อมูล (Subscribe)

การรับข้อมูลในระบบ MQTT จะรับข้อมูลได้เฉพาะเมื่อมีการเรียกใช้การ Subscribe ไปยัง Topic ที่กำหนด อาจเปรียบได้กับการ Subscribe คือการเข้าไปนั่งรอเพื่อนในกลุ่ม Line ส่งแชทมาหา เมื่อมีการส่งข้อมูลเข้ามาจะเกิดสิ่งที่เรียกว่าเหตุการณ์ (Event) ให้เรากดเข้าไปดูข้อความที่เพื่อน ๆ ส่งเข้ามา

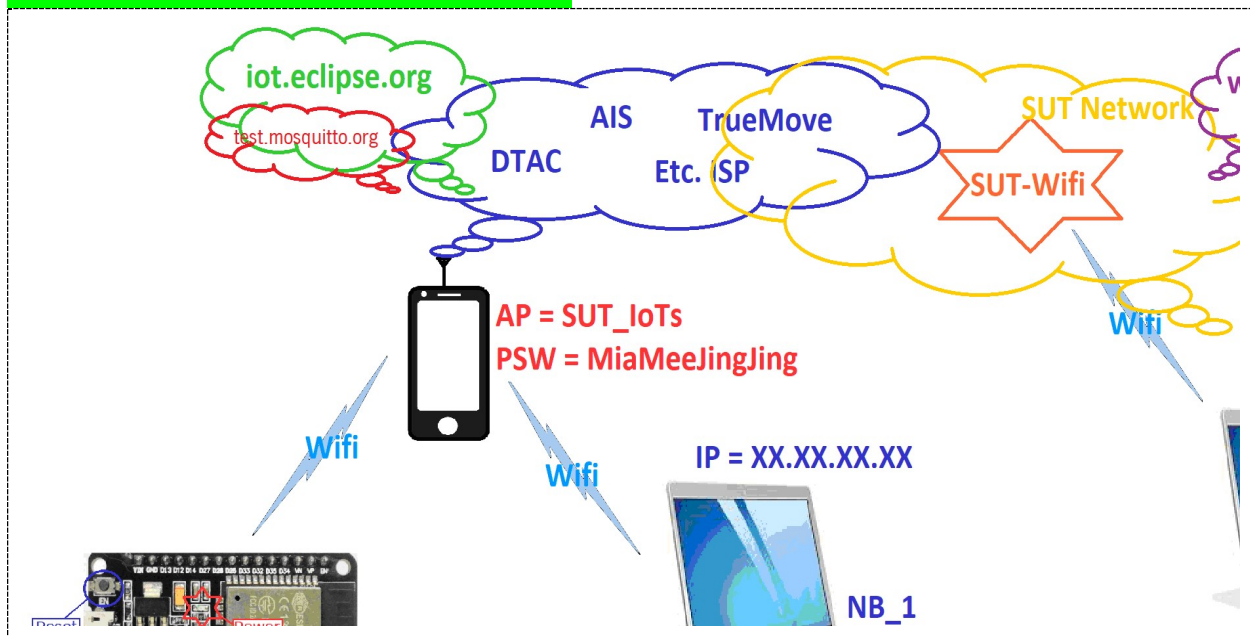
จะเห็นได้ว่า MQTT ก็เปลี่ยนเสมือนห้องแชทของอุปกรณ์ที่จะสนทนาแลกเปลี่ยนข้อมูลกันแบบเรียลไทม์ผ่านเครือข่ายอินเทอร์เน็ต

1.5 IoT มีวิธีการทำงานอย่างไร?

องค์ประกอบหลักของ IoT จะมี 3 ส่วนคือ **Broker**, **Publisher** และ **Subscriber**. ซึ่งการรับและส่งข้อมูลนั้น มันจะส่งข้อมูลไปมาหากันนั้นจะส่งผ่านตัวกลางนั้นก็คือ **Broker Server** โดยตัวส่งนี้จะเรียกว่า **Publisher** ส่งข้อมูลขึ้นไปยัง **Broker** พร้อมระบุหัวข้อ (Topic) ที่ต้องการส่งข้อมูลออกไป จากนั้นตัวรับซึ่งเรียกว่า **Subscriber** ถ้าหากตัวรับต้องการรับข้อมูลจากตัวส่งจะต้องทำการ **Subscribe** หัวข้อ Topic ของ **Publisher** นั้นๆ ผ่าน **Broker** เช่นกัน ลองดูความสัมพันธ์ ตามรูป



1.6 ข้อแตกต่างระหว่าง IoT กับ All Over IP



การทดลองก่อนหน้านี้เป็นการควบคุมผ่านอินเทอร์เน็ต จำเป็นต้องรู้ IP ของอุปกรณ์ปลายทาง และระบบต้องอยู่ในวงเครือข่ายเดียวกัน เช่น จากรูปเราไม่สามารถใช้ NB_2 เข้ามาควบคุม ESP32 ได้โดยตรงเพราะ IP=ZZ.ZZ.ZZ.ZZ และ IP=YY.YY.YY.YY อยู่คนละเครือข่าย หากต้องการสามารถกำหนดเส้นทางจาก NB_2 ผ่านเครือข่ายของมหาวิทยาลัย ไปยังผู้ให้บริการมือถือ วนมาที่มือถือ เข้ามายัง ESP32 การควบคุมสั่งการแบบนี้จำเป็นต้องรู้เลขปลายทางซึ่งเป็นไอพีของอุปกรณ์

กรณีของ IoTs กระบวนการข้างต้นจะปรับเปลี่ยน คือ ไม่จำเป็นต้องรู้เลขไอพีของอุปกรณ์ปลายทางแต่ให้อุปกรณ์วิ่งไปรับคำสั่งที่ตัวกลาง (Broker) แทน จากรูป NB_2 จะส่งคำสั่ง (Publish) ไปยังตัวกลาง ตัวอุปกรณ์ปลายทางต้องแจ้งรับข้อความ (Subscribe) จากตัวกลาง เมื่อมีคำสั่งเข้ามาและตัวอุปกรณ์ปลายทางเข้ามารับข้อมูลอุปกรณ์ปลายทางค่อยทำงานตามคำสั่งที่ได้รับ

เห็นได้ว่าแบบแรกจำเป็นต้องเข้าให้ถึง ESP32 แต่แบบหลังใช้วิธีนี้รับข้อความที่ตัวกลางที่ทั้งสองฝั่งตกลงกันได้

2a. การทดลอง

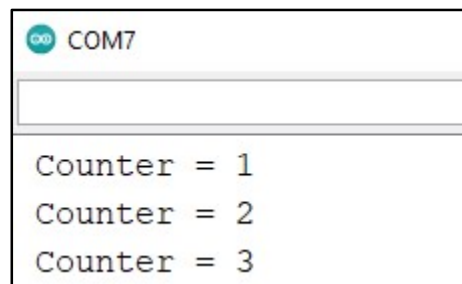
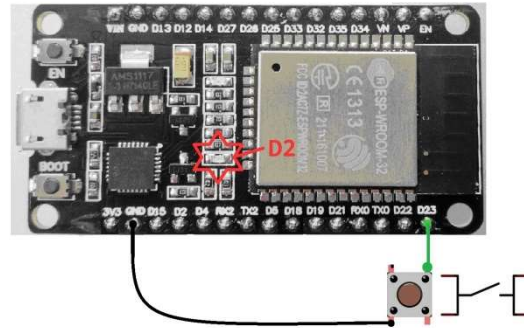
การทดลองที่ 1/4 Publish

- ทดสอบโปรแกรมนับจำนวนครั้งการกดสวิตช์ แสดงผลที่ Serial Monitor ด้วย baud rate 115200

```
int pushButton = 23;
int LED_Monitor = 2;
int Counter = 0;

void setup() {
  Serial.begin(115200);
  pinMode(LED_Monitor, OUTPUT);
  pinMode(pushButton, INPUT_PULLUP);
  Serial.print(" Counter = ");
  Serial.println(Counter);
  digitalWrite(LED_Monitor, Counter % 2);
}

void loop() {
  if (digitalRead(pushButton) == 0)
  { Counter++;
    Serial.print(" Counter = ");
    Serial.println(Counter);
    digitalWrite(LED_Monitor, Counter % 2);
    while (digitalRead(pushButton) == 0);
    delay(100);
  }
}
```



- Add Library → การใช้งาน MQTT บน ESP32 จะใช้งานผ่านไลบรารี PubSubClient.h จะต้องติดตั้งเพิ่มเติมโดยใช้ Library Manager **PubSubClient3 by Nick O'Leary Version 3.3.0**

PubSubClient3 by Nick O'Leary <nick.oleary@gmail.com>

A client library for MQTT messaging. MQTT is a lightweight messaging protocol ideal for small devices. This library allows you to send and receive MQTT messages. It supports the latest MQTT 3.1.1 protocol and can be configured to use the older MQTT 3.1 if needed. It supports all Arduino Ethernet Client compatible hardware, including the Intel...

[More info](#)

3.3.0

INSTALL

- โปรแกรมจะ Publish ไปยัง **test.mosquitto.org** ในหัวข้อ **myHome1234**

- Test Broker = **test.mosquitto.org**
- Port = **1883**
- Topic, Subscribe, Publish = **myHome1234**



<https://mntolia.com/10-free-public-private-mqtt-brokers-for-testing-prototyping/>
<https://www.hivemq.com/blog/mqtt-toolbox-mqtt-lens/>

5. ทดสอบด้วย MQTTCool >> https://mqtt.aarsoftwareserver.com:444/test_client/
หรือตัวอื่นๆ เช่น

- MQTTX Web: ใช้งานง่ายมากและรองรับ MQTT 5.0 >> <https://mqttx.app/web-client>
- HiveMQ Web Client: สำหรับทดสอบ Broker >> <https://www.hivemq.com/demos/websocket-client/>
- EMQX Dashboard: >> <https://www.emqx.com/en/mqtt/public-mqtt5-broker>
- ลอง MQTT Client Software >> "https://mqtt-explorer.com/"

- กำหนด Protocol = tcp, Host = test.mosquitto.org, Port = 1883
- Subscribe Topic, Publish Topic = myHome1234

Connection Settings for broker.mqtt.cool

Protocol: tcp Host: test.mosquitto.org Port: 1883

Username: The username Password: The password

☒ Shared Connection ☐ Dedicated Connection

Close Save changes

MQTT.Cool Test Client

testclient-cloud.mqtt.cool

Connection: tcp://test.mosquitto.org:1883

Subscriptions

The topic filter QoS 0 Subscribe

Subscribed topics

myHome1234 QoS 0

Publish

The destination of the mess QoS 0 Retain

The message text to be sent Publish

Messages

hello world #4

2023-7-19 8:25:14.530 topic: myHome1234 QoS 0

hello world #3

2023-7-19 8:25:8.377 topic: myHome1234 QoS 0

hello world #2

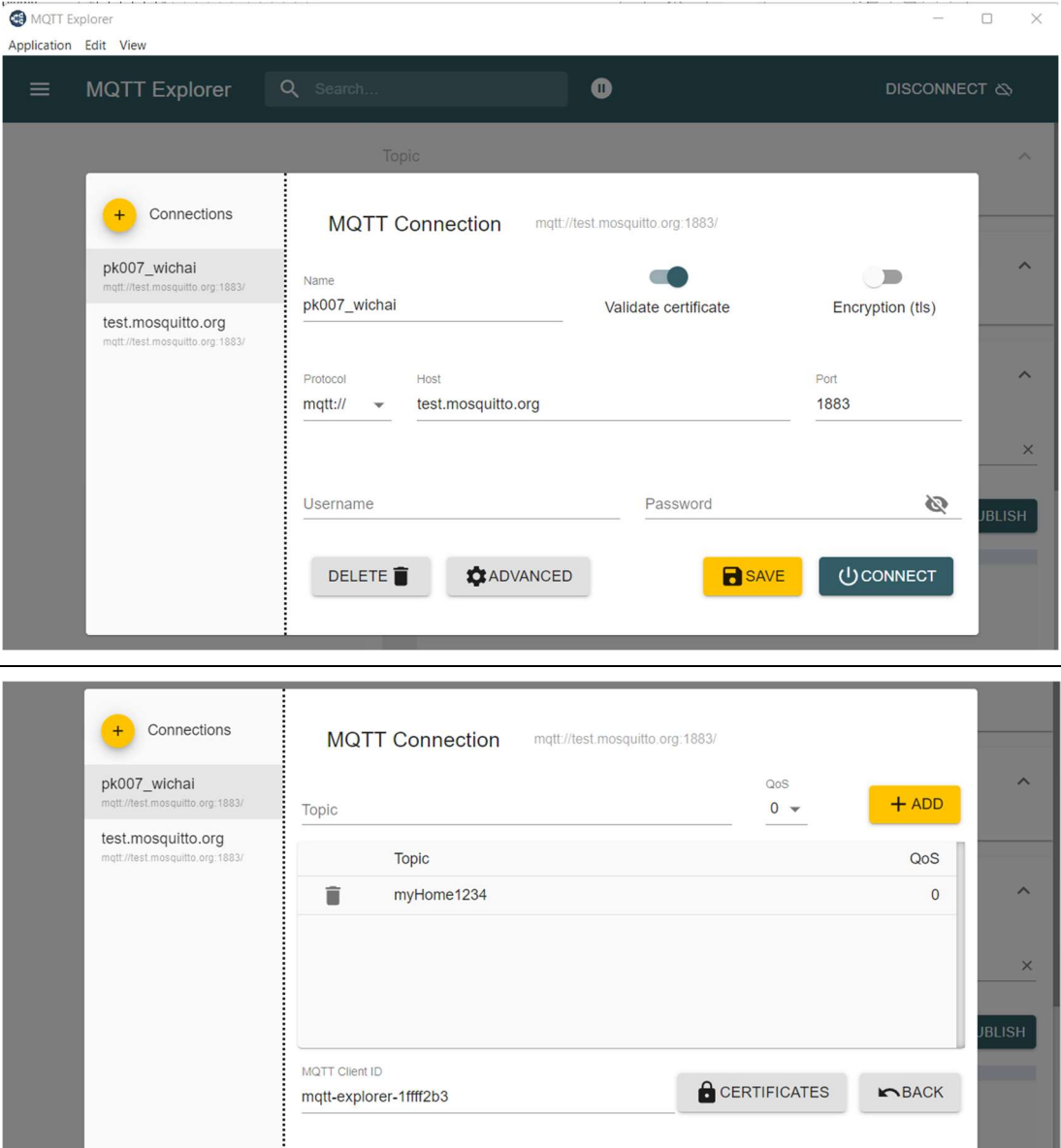
2023-7-19 8:25:7.624 topic: myHome1234 QoS 0

hello world #1

2023-7-19 8:25:6.350 topic: myHome1234 QoS 0

Hello World Pk007

5. ทดสอบด้วย MQTT Explorer Software >> <https://mqtt-explorer.com/>



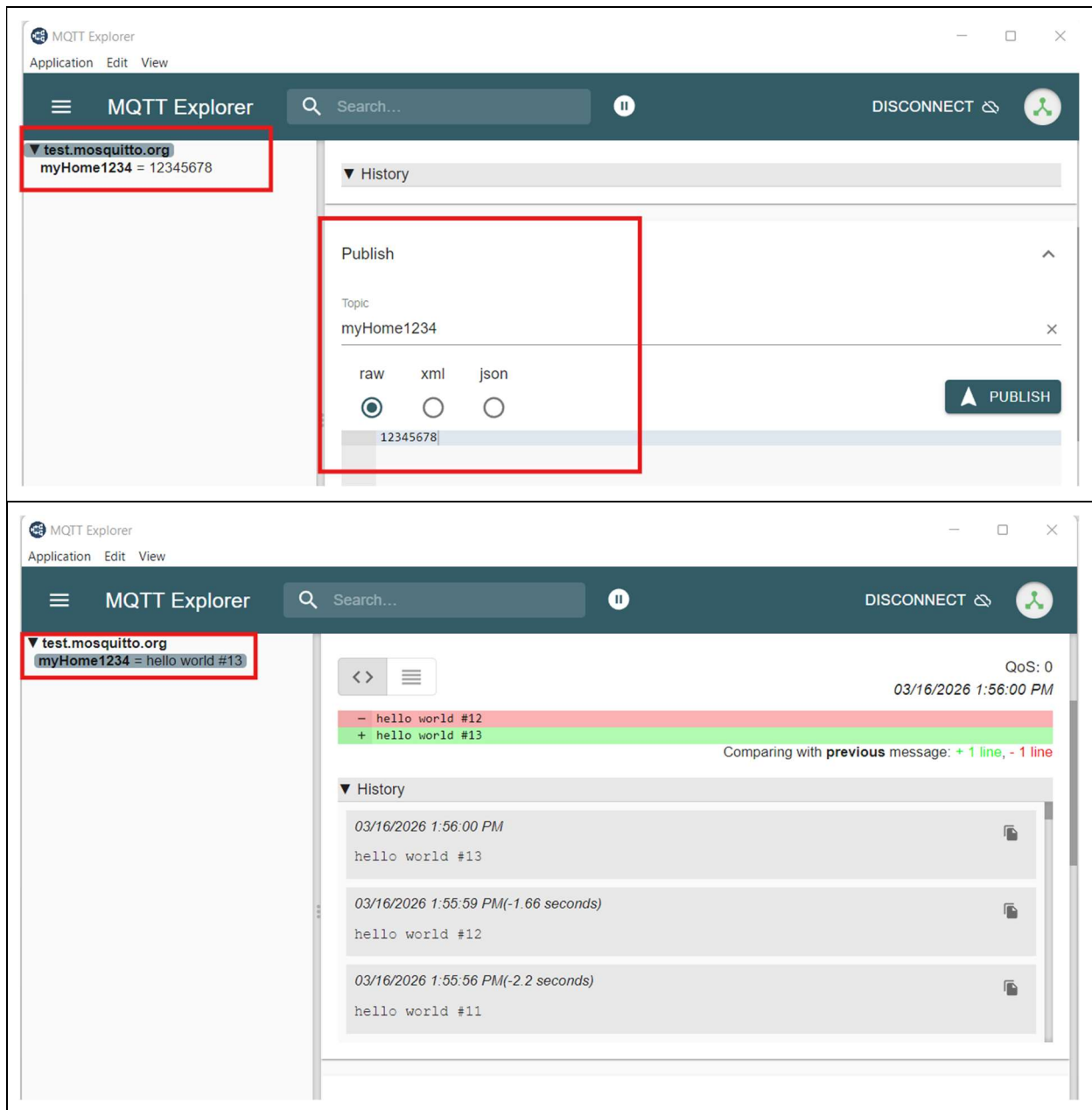
The top screenshot shows the MQTT Explorer interface with the 'MQTT Connection' configuration screen. The 'Connections' list on the left includes 'pk007_wichai' and 'test.mosquitto.org'. The configuration fields are as follows:

Field	Value
Name	pk007_wichai
Protocol	mqtt://
Host	test.mosquitto.org
Port	1883
Username	
Password	

The bottom screenshot shows the MQTT Explorer interface with the 'MQTT Connection' screen. The 'Connections' list on the left includes 'pk007_wichai' and 'test.mosquitto.org'. The configuration fields are as follows:

Field	Value
Topic	myHome1234
QoS	0
MQTT Client ID	mqtt-explorer-1ffff2b3

- กำหนด Protocol = mqtt, Host = test.mosquitto.org, Port = 1883
- Tab Advance เพื่อกำหนด Subscribe Topic
 - >> หากเป็น # คือรับทุก Topic
 - >> เพิ่ม Topic = myHome1234



- <https://mqtt-explorer.com/video.mp4>

6. โปรแกรมทดสอบ Publish ไปยัง **test.mosquitto.org** ในหัวข้อ **myHome1234**

```
#include <WiFi.h>
#include <PubSubClient.h>

const char* ssid = "SUT_IoTs";
const char* password = "MaiMeeJingJing";
const char* mqtt_server = "test.mosquitto.org";
const char* topic1 = "myHome1234";

WiFiClient espClient;
PubSubClient client(espClient);
long lastMsg = 0;
char msg[50];
int value = 0;

void setup_wifi() {
  delay(10);
  Serial.println();
  Serial.print("Connecting to ");
  Serial.println(ssid);
  WiFi.begin(ssid, password);
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }

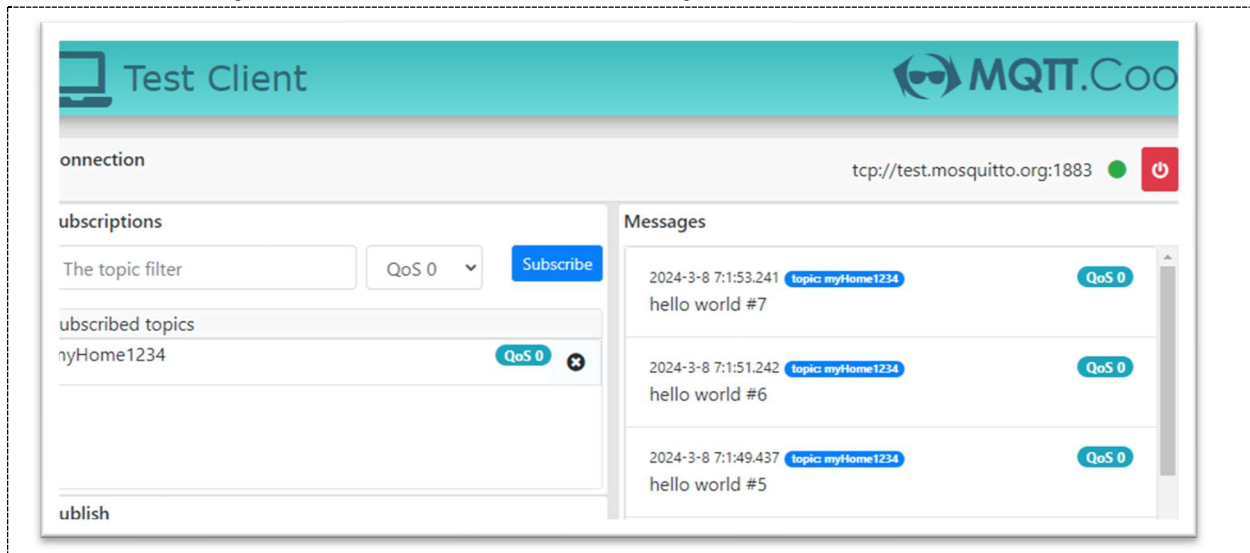
  randomSeed(micros());
  Serial.println("");
  Serial.println("WiFi connected");
  Serial.println("IP address: ");
  Serial.println(WiFi.localIP());
}

void reconnect()
{ while (!client.connected()) // Loop until we're reconnected
  { Serial.print("Attempting MQTT connection...");
    String clientId = "ESP32 Client-";
    clientId += String(random(0xffff), HEX); // Create a random client ID
    if (client.connect(clientId.c_str())) // Attempt to connect
    { Serial.println("connected"); // Once connected, publish an announcement...
      client.publish(topic1, "Hello World Pk007"); // ... and resubscribe
      client.subscribe(topic1);
    } else
    { Serial.print("failed, rc=");
      Serial.print(client.state());
      Serial.println(" try again in 5 seconds");
      delay(5000);
    }
  }
}

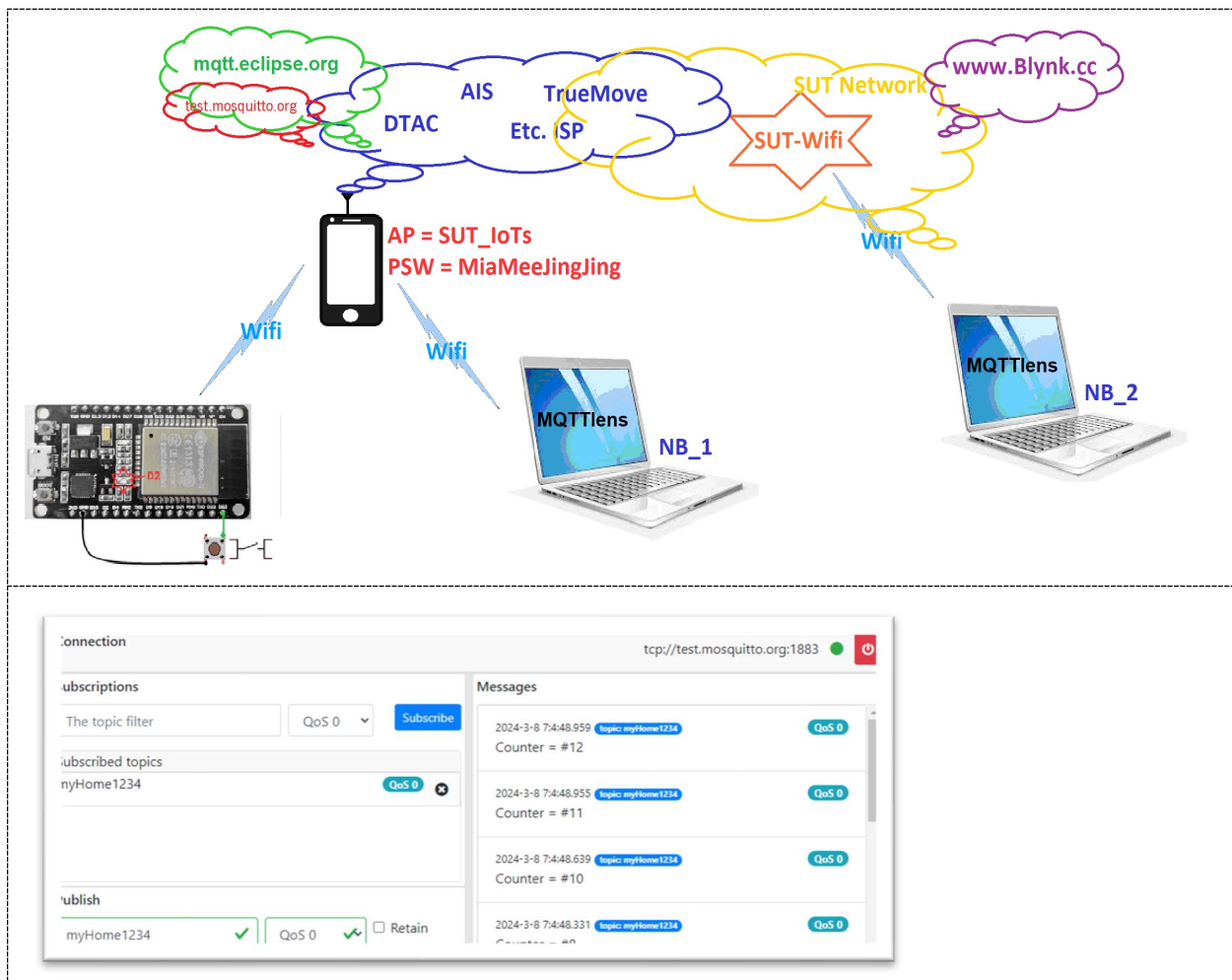
void setup()
{ Serial.begin(115200);
  setup_wifi();
  client.setServer(mqtt_server, 1883);
}

void loop()
{ if (!client.connected()) reconnect();
  client.loop();
  long now = millis();
  if (now - lastMsg > 2000)
  { lastMsg = now;
    ++value;
    snprintf (msg, 75, "hello world #%ld", value);
    Serial.print("Publish message: ");
    Serial.println(msg);
    client.publish(topic1, msg);
  }
}
```

7. ผลการทำงานดูที่ Serial Monitor ว่าส่งอะไรออกไป และดูที่ MQTT Cool ว่าได้รับอะไรมาบ้าง



8. ทดสอบโปรแกรมนับจำนวนครั้งการกดสวิทช์ ส่งค่าไปยัง MQTT Broker



9. โปรแกรมนับจำนวนครั้งการกดสวิตช์ ส่งค่าไปยัง MQTT Broker

```
#include <WiFi.h>
#include <PubSubClient.h>

const char* ssid = "SUT_IoTs";
const char* password = "MaiMeeJingJing";
const char* mqtt_server = "test.mosquitto.org";
const char* topic1 = "myHome1234";

WiFiClient espClient;
PubSubClient client(espClient);
long lastMsg = 0;
char msg[50];

int pushButton = 23;
int LED_Monitor = 2;
int Counter = 0;

void setup_wifi() {
  delay(10);
  Serial.println();
  Serial.print("Connecting to "); Serial.println(ssid);
  WiFi.begin(ssid, password);
  while (WiFi.status() != WL_CONNECTED) {
    delay(500); Serial.print(".");
  }
  randomSeed(micros());
  Serial.println(""); Serial.println("WiFi connected");
  Serial.println("IP address: "); Serial.println(WiFi.localIP());
}

void reconnect()
{ while (!client.connected()) // Loop until we're reconnected
  { Serial.print("Attempting MQTT connection...");
    String clientId = "ESP32 Client-";
    clientId += String(random(0xffff), HEX); // Create a random client ID
    if (client.connect(clientId.c_str())) // Attempt to connect
    { Serial.println("connected"); // Once connected, publish an announcement...
      client.publish(topic1, "Hello World Pk007"); // ... and resubscribe
      client.subscribe(topic1);
    } else
    { Serial.print("failed, rc=");
      Serial.print(client.state());
      Serial.println(" try again in 5 seconds");
      delay(5000);
    }
  }
}

void setup()
{ Serial.begin(115200);
  pinMode(LED_Monitor, OUTPUT);
  pinMode(pushButton, INPUT_PULLUP);
  setup_wifi();
  client.setServer(mqtt_server, 1883);
}

void loop()
{ if (digitalRead(pushButton) == 0)
  { Counter++;
    if (!client.connected()) reconnect();
    client.loop();
    snprintf(msg, 75, "Counter = #%ld", Counter);
    Serial.print("Publish message: ");
    Serial.println(msg);
    client.publish(topic1, msg);
    digitalWrite(LED_Monitor, Counter % 2);
    while (digitalRead(pushButton) == 0);
    delay(100);
  }
}
```

Question

Q101: ปรับแก้โปรแกรมให้อ่านค่า DHT22 แล้วส่งไปยัง MQTT Broker ทุกๆ 5 วินาที

การทดลองที่ 2/4 Publish and Subscribe

10. เมื่อโหลดโปรแกรมแล้วทำการทดสอบ publish “OFF” → Off LED และ “ON” → On LED

Publish

myHome1234 ✓ QoS 0 ✓

ON

Publish

myHome1234 ✓ QoS 0 ✓

OFF

Publish

Topic

myHome1234

raw xml json

☒ ☐ ☐

ON

Publish

Topic

myHome1234

raw xml json

☒ ☐ ☐

OFF

Test Client

MQTT.Co

Connection

tcp://test.mosquitto.org:1883

Subscriptions

The topic filter

QoS 0

Subscribe

Subscribed topics

myHome1234 QoS 0

Publish

myHome1234 ✓ QoS 0 ✓ ☐ Retain

OFF ✓

Publish

Messages

2024-3-8 7:7:26.812 topic: myHome1234 QoS 0

OFF

2024-3-8 7:6:44.411 topic: myHome1234 QoS 0

ON

2024-3-8 7:4:48.959 topic: myHome1234 QoS 0

Counter = #12

2024-3-8 7:4:48.955 topic: myHome1234 QoS 0

Counter = #11

2024-3-8 7:4:48.639 topic: myHome1234 QoS 0

11. โปรแกรมการ Publish และ Subscribe หน่วงเวลาครั้งละ 5 วินาที

```

#include <WiFi.h>
#include <PubSubClient.h>

const char* ssid = "GUT_IoTs";
const char* password = "MaiMeeJingJing";
const char* mqtt_server = "test.mosquitto.org";
const char* topic1 = "myHome1234";

WiFiClient espClient;
PubSubClient client(espClient);
long lastMsg = 0;
char msg[50];

int TestLED = 2;
int value = 0;

void setup_wifi() {
  delay(10);
  Serial.println();      Serial.print("Connecting to ");
  Serial.println(ssid);   WiFi.begin(ssid, password);
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);          Serial.print(".");
  }
  randomSeed(micros());
  Serial.println("");      Serial.println("WiFi connected");
  Serial.println("IP address: ");   Serial.println(WiFi.localIP());
  pinMode(TestLED, OUTPUT);
}

void callback(char* topic, byte* payload, unsigned int length)
{ char myPayload[50];
  Serial.print("Message arrived ["); Serial.print(topic1);          Serial.print("] ");
  for (int i = 0; i < length; i++)
  { Serial.print((char)payload[i]);
    myPayload[i] = payload[i];
    myPayload[i + 1] = '\0'; // End of String
  }
  Serial.print("\n ---> "); Serial.println(myPayload);
  myPayload[4] = '\0'; // String less than 4 Charector
  if ((String)myPayload == "ON") digitalWrite(TestLED, HIGH);
  if ((String)myPayload == "OFF") digitalWrite(TestLED, LOW);
}

void reconnect()
{ while (!client.connected()) // Loop until we're reconnected
  { Serial.print("Attempting MQTT connection...");
    String clientId = "ESP8266Client-";
    clientId += String(random(0xffff), HEX); // Create a random client ID
    if (client.connect(clientId.c_str())) // Attempt to connect
    { Serial.println("connected"); // Once connected, publish an announcement...
      client.publish(topic1, "Hello World Pk007"); // ... and resubscribe
      client.subscribe(topic1);
    } else
    { Serial.print("failed, rc=");
      Serial.print(client.state());
      Serial.println(" try again in 5 seconds");
      delay(5000);
    }
  }
}

void setup()
{ Serial.begin(115200);
  setup_wifi();
  client.setServer(mqtt_server, 1883);
  client.setCallback(callback);
  pinMode(TestLED, OUTPUT);
}

void loop()
{ if (!client.connected()) reconnect();
  client.loop();
  long now = millis();
  if (now - lastMsg > 2000)
  { lastMsg = now;
    ++value;
    sprintf(msg, "hello world #%ld", value);
    Serial.print("Publish message: ");      Serial.println(msg);
    client.publish(topic1, msg);
  }
}

```

13. ทดสอบโปรแกรมนับจำนวนครั้งการกดสวิตช์ ส่งค่าไปยัง MQTT Broker และควบคุมการปิด-เปิด LED

```
// DOIT ESP-32

#include <WiFi.h>
#include <PubSubClient.h>

const char* ssid = "SUT_ToTs";
const char* password = "MaiMeeJingJing";
const char* mqtt_server = "test.mosquitto.org";
const char* topic1 = "myHome1234";

WiFiClient espClient;
PubSubClient client(espClient);
long lastMsg = 0;
char msg[50];

int TestLED = 2;
int pushButton = 23;
int Counter = 0;

void setup_wifi() {
  delay(10);
  Serial.println();
  Serial.print("Connecting to ");
  Serial.println(ssid);
  WiFi.begin(ssid, password);
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }

  randomSeed(micros());
  Serial.println("");
  Serial.println("WiFi connected");
  Serial.println("IP address: ");
  Serial.println(WiFi.localIP());
}

void callback(char* topic, byte* payload, unsigned int length)
{
  char myPayload[50];
  Serial.print("Message arrived [");
  Serial.print(topic);
  Serial.print("] ");
  for (int i = 0; i < length; i++)
  { Serial.print((char)payload[i]);
    myPayload[i] = payload[i];
    myPayload[i + 1] = '\0'; // End of String
  }
  Serial.print("\n ---> "); Serial.println(myPayload);
  myPayload[4] = '\0'; // String less than 4 Charector
  if ((String)myPayload == "ON") digitalWrite(TestLED, HIGH);
  if ((String)myPayload == "OFF") digitalWrite(TestLED, LOW);
}

void reconnect()
{
  while (!client.connected()) // Loop until we're reconnected
  { Serial.print("Attempting MQTT connection...");
    String clientId = "ESP32 Client-";
    clientId += String(random(0xffff), HEX); // Create a random client ID
    if (client.connect(clientId.c_str())) // Attempt to connect
    { Serial.println("connected"); // Once connected, publish an announcement...
      client.publish(topic1, "Hello World Pk007"); // ... and resubscribe
      client.subscribe(topic1);
    } else
    { Serial.print("failed, rc=");
      Serial.print(client.state());
      Serial.println(" try again in 5 seconds");
      delay(5000);
    }
  }
}

void setup()
{ Serial.begin(115200);
  pinMode(pushButton, INPUT_PULLUP);
  pinMode(TestLED, OUTPUT);
  setup_wifi();
  client.setServer(mqtt_server, 1883);
  client.setCallback(callback);
}
```

```

void loop()
{
  if (!client.connected()) reconnect();
  client.loop();
  if (digitalRead(pushButton) == 0)
  {
    Counter++;
    client.loop();
    sprintf (msg, "Count = %d", Counter);
    Serial.print("Publish message: ");
    Serial.println(msg);
    client.publish(topic1, msg);
    while (digitalRead(pushButton) == 0);
    delay(100);
  }
}

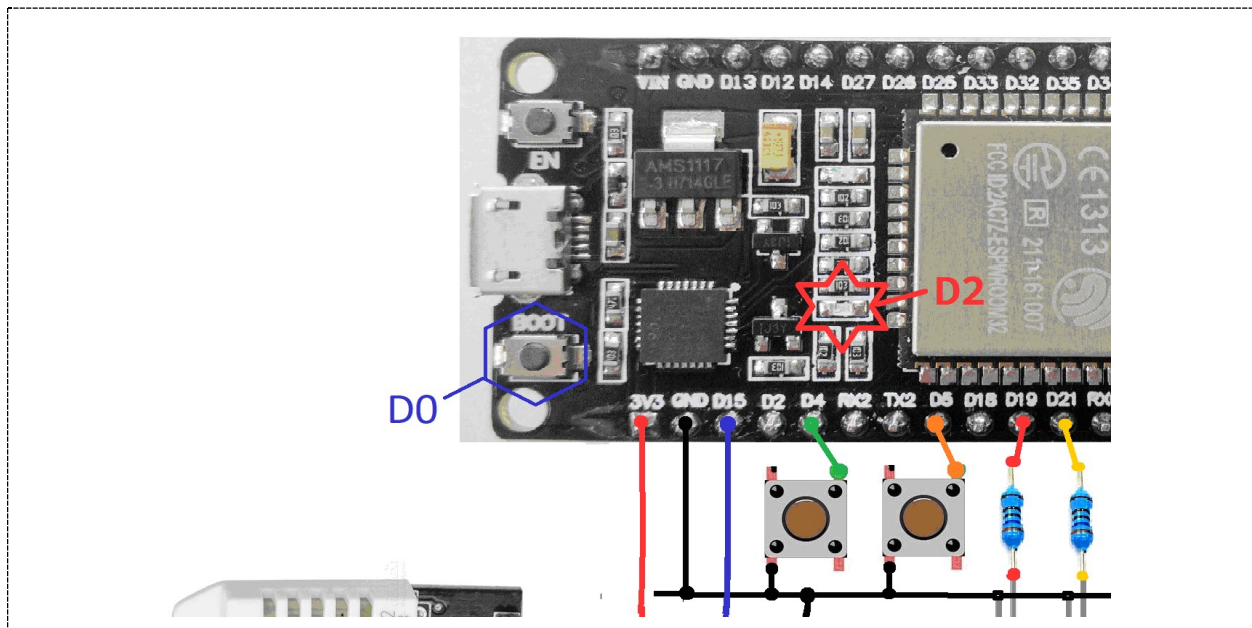
```

Question

Q102: อ่านค่า DHT-22 แล้วส่งไปยัง MQTT Broker ทุกๆ 5 วินาที และควบคุมการปิด-เปิด LED 4 ตัว

Q103: อ่านค่า DHT-22 แล้วส่งไปยัง MQTT Broker ทุกๆ 5 วินาที + ควบคุม 4 LED + รับค่าสวิตช์กำหนด

- SW1 แจ้ง Overheat Alarm
- SW2 แจ้ง Intruders Alarm



2/3: Ubidots IoT Platform -- using Ubidots STEM -- (Update 20260315)

การทดลองที่ 3/4: Ubidots Platform

Read More - Test with MQTTLens

- <https://youtu.be/37jVqKN1IJs>
- <https://ubidots.com/docs/hw/#mqtt>
- <https://automation360blog.wordpress.com/2018/08/13/ubidots/>

Exp3_1/5: Start Ubidots and MQTT Client to Ubidots

1. Ubidots STEM -- <https://ubidots.com/stem>



STEM
LOG IN

CREATE STEM ACCOUNT

ขั้นตอนการสมัคร Ubidots STEM

1. ไปที่หน้าเว็บสำหรับ STEM โดยเฉพาะ:
เข้าสู่เว็บไซต์ stem.ubidots.com (ย้ำว่าต้องมีคำว่า **stem** นะครับ ถ้าไปหน้าหลัก ubidots.com ปกติจะเจอปุ่มทดลองใช้ฟรี 30 วันของเวอร์ชันธุรกิจแทน)
2. เลือก Create Account:
คลิกที่ปุ่ม "Create an Account" หรือ "Sign Up"
3. กรอกข้อมูลส่วนตัว:
 - **Username:** ตั้งชื่อผู้ใช้งาน
 - **Email:** ใช้อีเมลที่ใช้งานจริง (ต้องเข้าไปกดยืนยันในภายหลัง)
 - **Password:** ตั้งรหัสผ่านที่ปลอดภัย
4. ยอมรับเงื่อนไข:
ติ๊กถูกที่ช่องยอมรับข้อตกลงการใช้งาน (Terms of Service) แล้วกดปุ่ม "Sign up for free"
5. ยืนยันอีเมล:
เช็คกล่องจดหมายในอีเมลของคุณ จะมีลิงก์จาก Ubidots ส่งไปให้กด "Verify your email" เพื่อเปิดใช้งานบัญชีอย่างสมบูรณ์

 Create a free account

pk

wichai

wichai@g.sut.ac.th

pk009

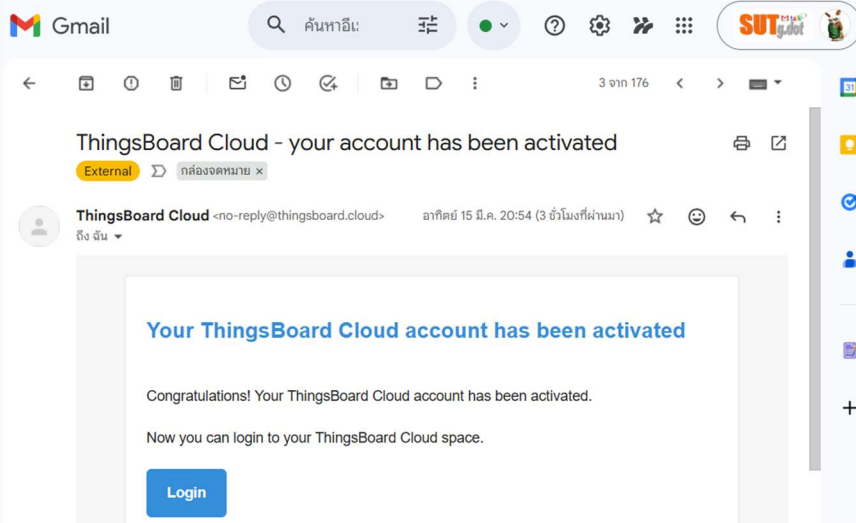
••••••••



MyIoT project is for personal, non-commercial use.



I'm not a robot

reCAPTCHA
Privacy - Terms**Sign Up**

ข้อต้องระวังในการใช้งาน Ubidots STEM

การใช้งาน Ubidots STEM ซึ่งเป็นเวอร์ชันฟรีสำหรับนักพัฒนาและผู้เริ่มต้น มีข้อจำกัดและข้อควรระวังที่สำคัญเพื่อให้คุณใช้งานได้อย่างต่อเนื่องโดยไม่โดนระงับบัญชีหรือข้อมูลสูญหาย ดังนี้ครับ:

1. ข้อจำกัดด้านจำนวนอุปกรณ์และข้อมูล (Usage Limits)

Ubidots STEM มีเพดานการใช้งานที่ค่อนข้างเข้มงวด หากเกินขีดจำกัด ระบบอาจจะหยุดรับข้อมูลทันที:

- Devices: เชื่อมต่ออุปกรณ์ได้สูงสุด 3 เครื่อง
- Variables: สร้างตัวแปรได้สูงสุด 10 ตัว (รวมทุก Device)
- Data Retention: ข้อมูลจะถูกเก็บไว้เพียง 1 เดือน เท่านั้น (ถ้าต้องการเก็บประวัตินานกว่านั้นต้องสำรองข้อมูลเอง)
- Dots Per Day: ส่งข้อมูลได้ไม่เกิน 4,000 Dots ต่อวัน (1 Dot = 1 ค่าที่ส่งขึ้นไป)

2. อัตราการส่งข้อมูล (Ingestion Rate)

อย่าส่งข้อมูลถี่เกินไป!

- Rate Limit: แนะนำให้ส่งข้อมูลทุกๆ 1-5 นาที ต่อหนึ่งตัวแปร
- ผลกระทบ: หากส่งข้อมูลเร็วเป็นวินาที (Spamming) บัญชีของคุณอาจถูกมองว่าเป็น Bot และอาจถูกแบนชั่วคราว หรือ API Key จะถูกระงับการใช้งาน (Throttling)

3. การรักษาความปลอดภัยของ Token

- Default Token: อย่าเปิดเผย Default Token ของคุณในที่สาธารณะ (เช่น GitHub หรือโพสต์ตามในกลุ่ม Facebook) เพราะใครที่มี Token นี้จะสามารถดึงข้อมูลหรือลบข้อมูลใน Dashboard ของคุณได้ทั้งหมด
- Regenerate: หากเผลอทำหลุด ให้รีบกด Regenerate Token ใหม่ทันทีในเมนู API Credentials

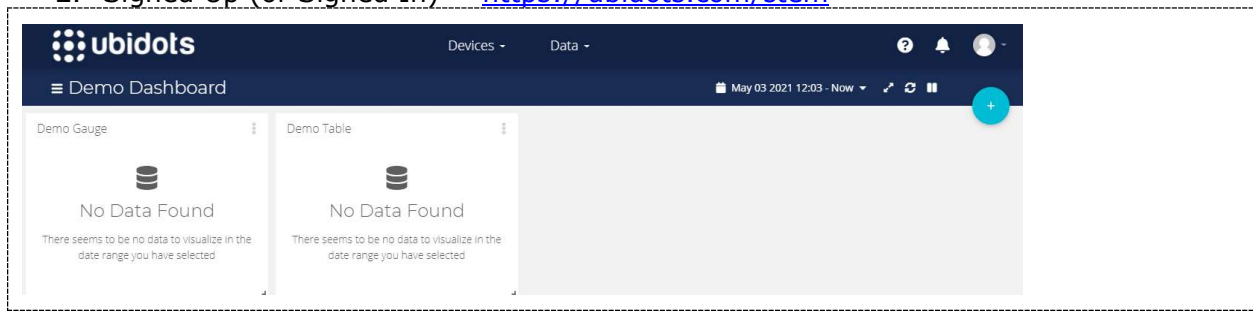
4. ข้อห้ามด้านพาณิชย์ (Non-Commercial Use)

- Personal Use Only: บัญชี STEM ออกแบบมาเพื่อการเรียนรู้ การวิจัย หรือโปรเจกต์ส่วนตัวเท่านั้น ห้ามนำไปใช้ในเชิงธุรกิจหรือบริการลูกค้า หากระบบตรวจพบการใช้งานที่ดูเหมือนการทำธุรกิจ (เช่น มีการดึงข้อมูลหนักๆ จากหลายไอพี) บัญชีอาจถูกปิดได้

5. การจัดเรียง Dashboard

- Widget Limit: ในเวอร์ชันฟรี จำนวน Widget บน Dashboard อาจมีจำกัด และบาง Widget ขั้นสูง (Advanced Widgets) จะใช้งานไม่ได้
- Mobile App: การแสดงผลบนแอปมือถือของเวอร์ชัน STEM อาจไม่ลื่นไหลหรือปรับแต่งได้เท่าเวอร์ชันเสียเงิน

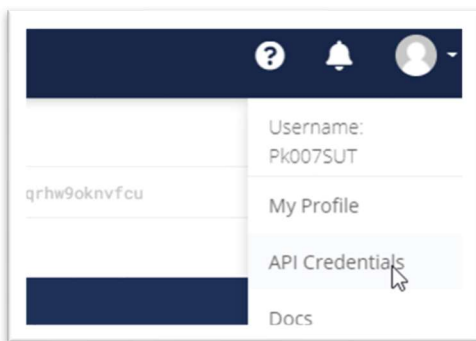
2. Signed Up (or Signed In) -- <https://ubidots.com/stem>



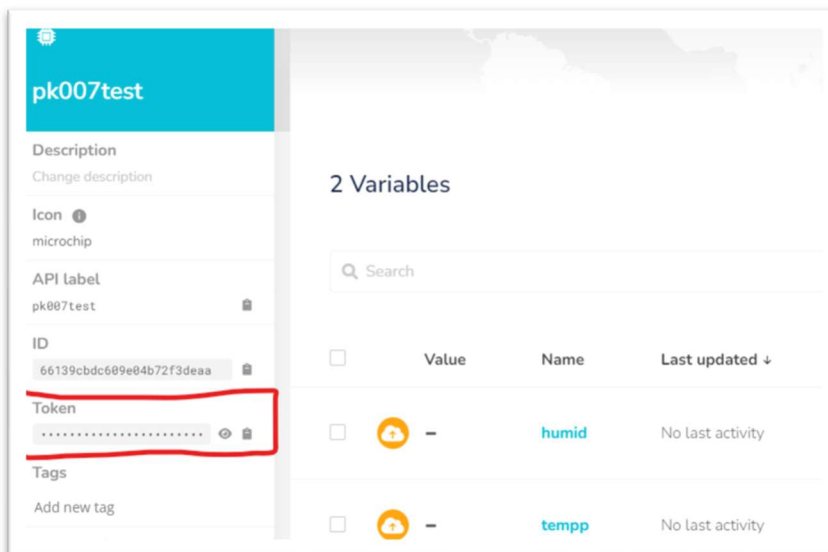
3. Create Device and Variable

	Device --> Device
Device name pk007test Device label pk007test	Create Device Blank Device Device Name = pk007test Device Label = pk007test <all lowercase alphabet>
	Click pk007test Device Add Variable humid tempp <all lowercase alphabet>
	Click ... Set API Label = humid Set API Label = tempp <all lowercase alphabet>

4. Get Tokens Key



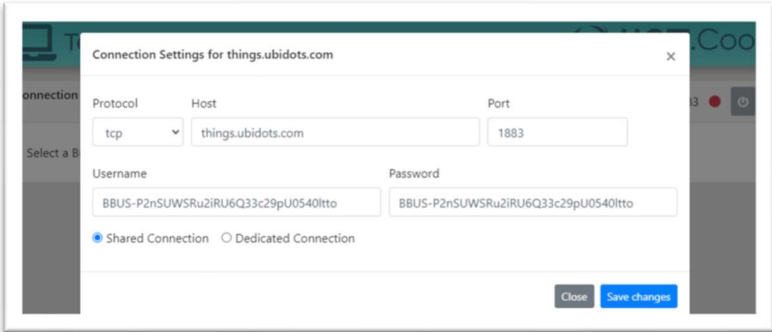
API Credentials



Copy Data

API Label: **pk007test**
ID: **6614b72f3deaa**
Token: **33c290540Itto**

5. MQTT Cool >> https://mqtt.aarsoftwareserver.com:444/test_client/



Start MQTT Cool

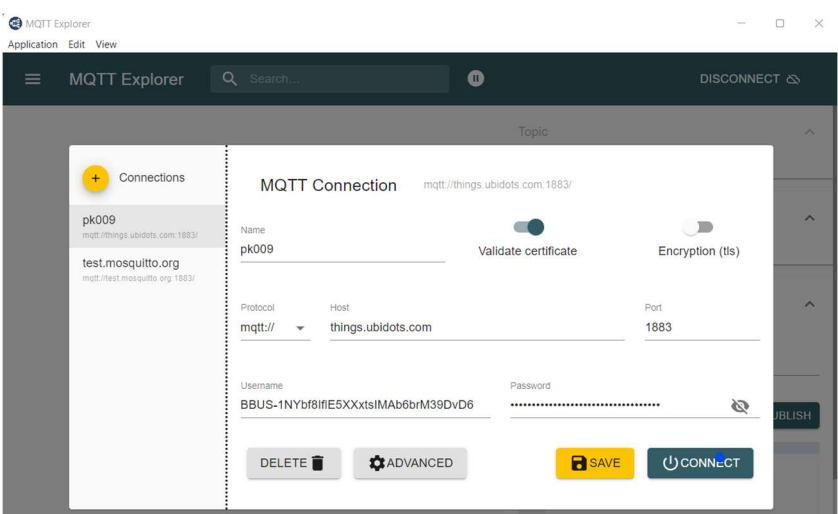
Host =
industrial.api.ubidots.com

Port =
1883

UserName = Tokens Key
Password = Tokens Key

Connect

- 5B. กรณี MQTT Client Software >> “<https://mqtt-explorer.com/>”



Name = **pk009**

Protocol =
mqtt://

Host =
industrial.api.ubidots.com

Port =
1883

UserName = Tokens Key
Password = Tokens Key

วิธีการ Subscribe ใน MQTT Explorer จะมีความพิเศษกว่าโปรแกรมอื่นเล็กน้อย ดังนี้ครับ:

1. การ Subscribe แบบอัตโนมัติ (Default)

เมื่อคุณกด Connect เข้าไปยัง Broker ครั้งแรก MQTT Explorer จะทำการ Subscribe ไปที่ Topic # (เครื่องหมายสี่เหลี่ยม) ให้โดยอัตโนมัติ

- # (Wildcard): หมายถึง "ขอรับข้อมูลทุกอย่างที่วิ่งอยู่ใน Broker นี้"
- ผลลัพธ์: คุณจะเห็น Topic ต่างๆ ค่อยๆ ปรากฏขึ้นที่แถบด้านซ้ายมือตามที่มีการส่งข้อมูลเข้ามาจริง

2. การกำหนด Topic ที่ต้องการเจาะจง (Custom Subscribe)

หาก Broker มีข้อมูลเยอะเกินไปจนเครื่องค้าง หรือคุณต้องการดูเฉพาะ Topic ที่สนใจ ให้ตั้งค่าก่อนกด Connect ดังนี้:

1. เปิดโปรแกรมและเลือก Connection ที่ต้องการ
2. คลิกที่ปุ่ม Advanced (อยู่มุมขวาล่าง หรือข้างๆ ปุ่ม Connect)
3. ในช่อง Subscribe:
 - ลบ # ออก (หากไม่ต้องการรับทุกอย่าง)
 - พิมพ์ Topic ที่ต้องการ เช่น home/livingroom/temp
 - กดปุ่ม Add
4. กด Back แล้วกด Connect

6. Subscribe/Publish Test

Connection

Subscriptions

The topic filter

QoS 0

Subscribe

Subscribed topics

/v2.0/devices/pk007test/tempp

QoS 0

/v2.0/devices/pk007test/humid

QoS 0

Subscribe Device / Variable

/v1.6/devices/pk007test/humid

Subscribe Device / Variable

/v1.6/devices/pk007test/tempp

Connections

pk007

mqtt://industrial.api.ubidots.com/...

MQTT Connection

mqtt://industrial.api.ubidots.com:1883/

Topic

/v1.6/devices/pk007test/humid

QoS

0

+ ADD

Topic

QoS

/v1.6/devices/pk007test/tempp

0

/v1.6/devices/pk007test/humid

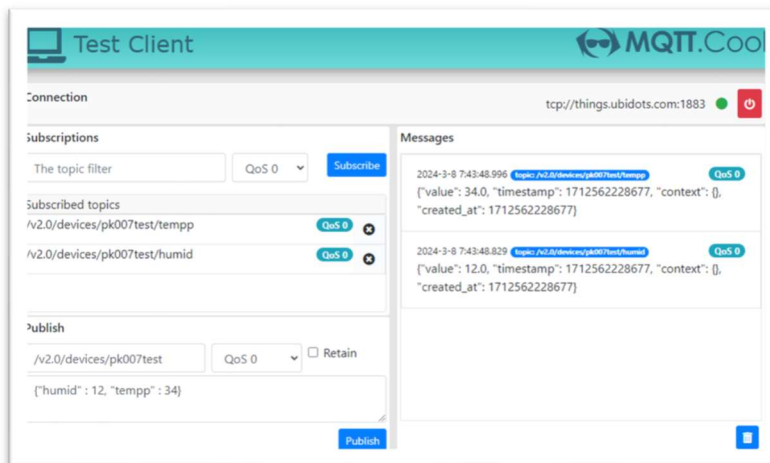
0

MQTT Client ID

mqtt-explorer-4b9fd457

CERTIFICATES

BACK



Publish

`/v1.6/devices/pk007test`

`{"humid" : 12, "temp" : 34}`

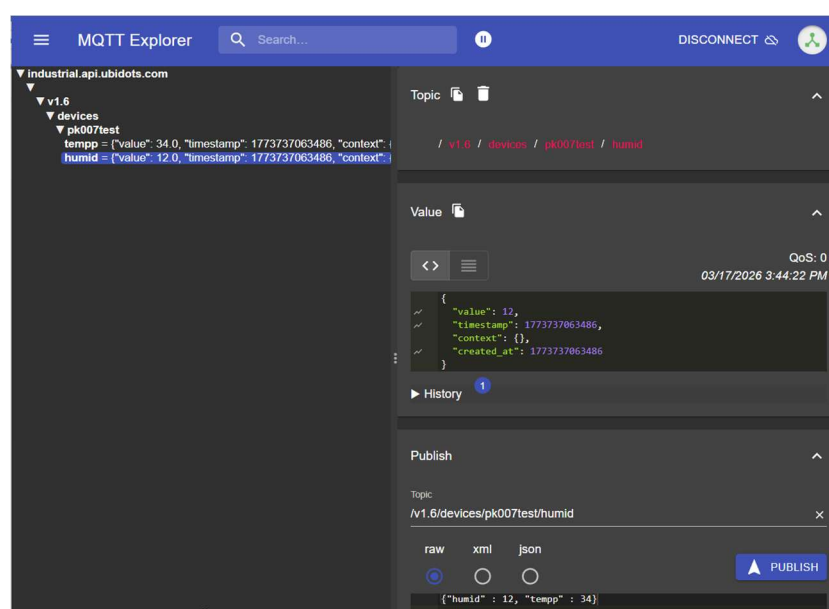
Response humid

Response temp

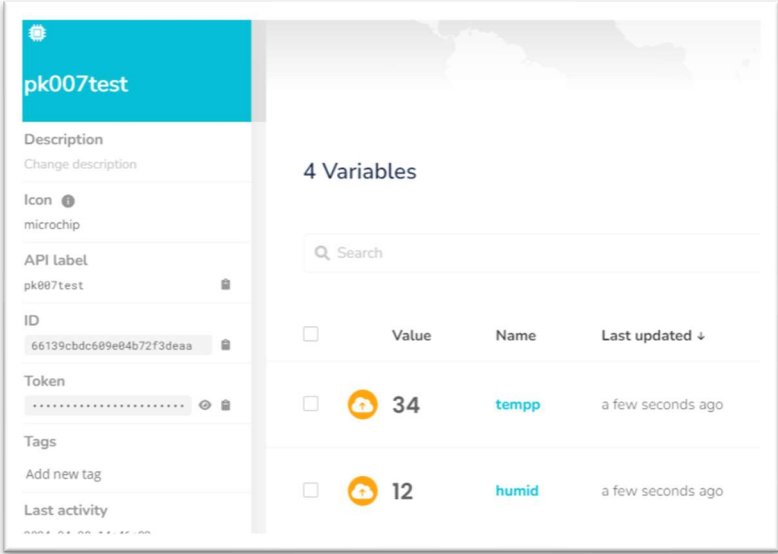
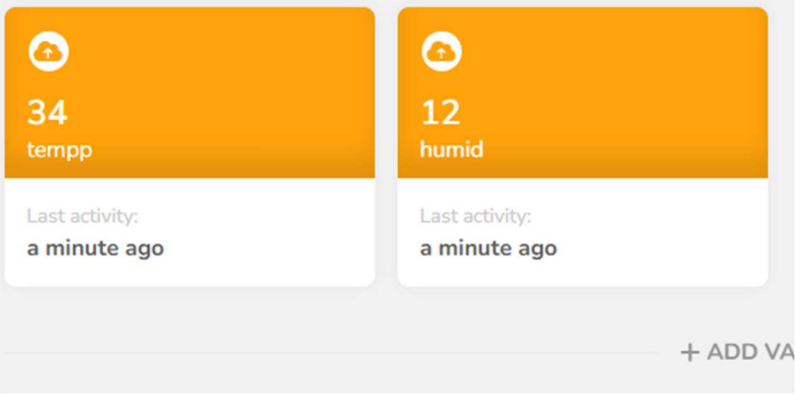


Publish via MQTT Explorer

Result



7. Result

	Show pk007test humid = 12, temp=34
	

Exp3_2/5: ESP32 Publish to Ubidots with MQTT Protocol

8. Prepare - make sure for esp32 library
 - 8.1 DHT22 Library

DHT sensor library for ESPx by beegee_tokyo Ver 1.19.0

DHT sensor library for ESPx by beegee_tokyo

Arduino ESP library for DHT11, DHT22, etc Temp & Humidity Sensors Optimized library to match ESP32 requirements. Last changes: Fix negative temperature problem (credits @helijunky)

[More info](#)

1.19.0 ▾

INSTALL

8.2 PubSub Client

PubSubClient3 by Nick O'Leary – V3.3.0

PubSubClient3 by Nick O'Leary <nick.oleary@gmail.com>

A client library for MQTT messaging. MQTT is a lightweight messaging protocol ideal for small devices. This library allows you to send and receive MQTT messages. It supports the latest MQTT 3.1.1 protocol and can be configured to use the older MQTT 3.1 if needed. It supports all Arduino Ethernet Client compatible hardware, including the Intel...

[More info](#)

3.3.0 ▾

INSTALL

9. Code for ESP32 Publish Random Data to Ubidots

```
#include <WiFi.h>
#include <PubSubClient.h>

const char *My_SSID = "IoT-Test";
const char *My_Pass = "pk12345678";

const char *MQTT_Server = "industrial.api.ubidots.com";
const char *MQTT_User = "BBUS-Vpmy8LzjBFQQireUISQIojhRS2vdNF";
const char *MQTT_Pass = "BBUS-Vpmy8LzjBFQQireUISQIojhRS2vdNF";
const char *PTopic1 = "/v1.6/devices/pk007test";
const char *STopic1 = "/v1.6/devices/pk007test/humid";
const char *STopic2 = "/v1.6/devices/pk007test/tempp";
#define MQTT_Port 1883

WiFiClient espClient;
PubSubClient client(espClient);
long lastMsg = 0;
char msg[50];

void Setup_Wifi() {
  delay(10);
  Serial.println();
  Serial.print("Connecting to ");
  Serial.println(My_SSID);
  WiFi.begin(My_SSID, My_Pass);
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);    Serial.print(".");
  }
  randomSeed(micros());
  Serial.println("");
  Serial.println("WiFi connected");
  Serial.println("IP address: ");
  Serial.println(WiFi.localIP());
}
```

```

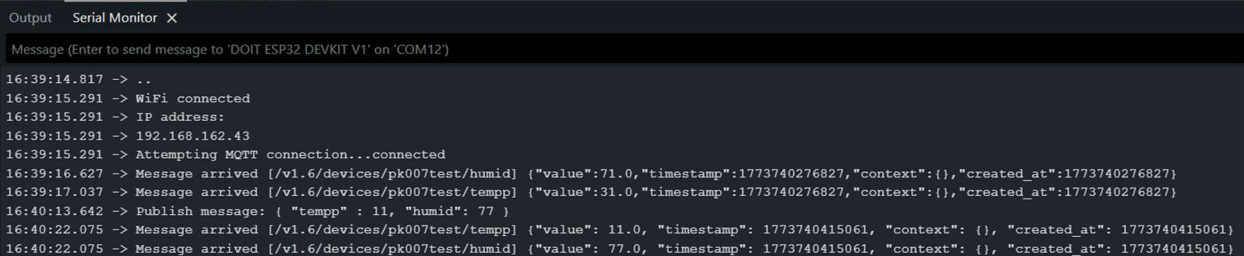
void reconnect() {
  while (!client.connected()) // Loop until we're reconnected
  {
    Serial.print("Attempting MQTT connection...");
    String clientId = "ESP32 Client-";
    clientId += String(random(0xffff), HEX); // Create a random client ID
    if (client.connect(clientId.c_str(), MQTT_User, MQTT_Pass)) // Attempt to connect
    {
      Serial.println("connected"); // Once connected, publish an announcement...
      client.subscribe(STopic1);
      client.subscribe(STopic2);
    } else {
      Serial.print("failed, rc=");
      Serial.print(client.state());
      Serial.println(" try again in 5 seconds");
      delay(5000);
    }
  }
}

void callback(char *topic, byte *payload, unsigned int length) {
  Serial.print("Message arrived [");
  Serial.print(topic);
  Serial.print("] ");
  for (int i = 0; i < length; i++) {
    Serial.print((char)payload[i]);
  }
  Serial.println();
}

void setup() {
  Serial.begin(115200);
  Setup_Wifi();
  client.setServer(MQTT_Server, MQTT_Port);
  client.setCallback(callback);
}

void loop() {
  if (!client.connected()) reconnect();
  client.loop();
  long now = millis();
  if (now - lastMsg > 60000) {
    lastMsg = now;
    int valueT = random(50);
    int valueH = random(50) + 50;
    snprintf(msg, 75, "{ \"temp\" : %ld, \"humid\" : %ld }", valueT, valueH);
    Serial.print("Publish message: ");
    Serial.println(msg);
    client.publish(PTopic1, msg);
  }
}

```



```

Output Serial Monitor X
Message (Enter to send message to 'DOIT ESP32 DEVKIT V1' on 'COM12')

16:39:14.817 -> ..
16:39:15.291 -> WiFi connected
16:39:15.291 -> IP address:
16:39:15.291 -> 192.168.162.43
16:39:15.291 -> Attempting MQTT connection...connected
16:39:16.627 -> Message arrived [/v1.6/devices/pk007test/humid] {"value":71.0,"timestamp":1773740276827,"context":{},"created_at":1773740276827}
16:39:17.037 -> Message arrived [/v1.6/devices/pk007test/temp] {"value":31.0,"timestamp":1773740276827,"context":{},"created_at":1773740276827}
16:40:13.642 -> Publish message: { "temp" : 11, "humid": 77 }
16:40:22.075 -> Message arrived [/v1.6/devices/pk007test/temp] {"value": 11.0, "timestamp": 1773740415061, "context": {}, "created_at": 1773740415061}
16:40:22.075 -> Message arrived [/v1.6/devices/pk007test/humid] {"value": 77.0, "timestamp": 1773740415061, "context": {}, "created_at": 1773740415061}

```

Test Client

MQTT.Cool

Connection

tcp://things.ubidots.com:1883

Subscriptions

The topic filter

QoS 0

Subscribe

Subscribed topics

/v2.0/devices/pk007test/tempp

QoS 0

/v2.0/devices/pk007test/humid

QoS 0

Messages

2024-3-8 7:53:12.795

topic: /v2.0/devices/pk007test/tempp

QoS 0

{"value": 31.0, "timestamp": 1712562792776, "context": {}, "created_at": 1712562792776}

2024-3-8 7:53:12.794

topic: /v2.0/devices/pk007test/humid

QoS 0

{"value": 20.0, "timestamp": 1712562792776, "context": {}, "created_at": 1712562792776}

MQTT Explorer

Search...

industrial.api.ubidots.com

v1.6

devices

pk007test

tempp = {"value": 13.0, "timestamp": 1773737797987, "context": {}}

humid = {"value": 67.0, "timestamp": 1773737797987, "context": {}}

67

humid

Last activity:

a few seconds ago

13

tempp

Last activity:

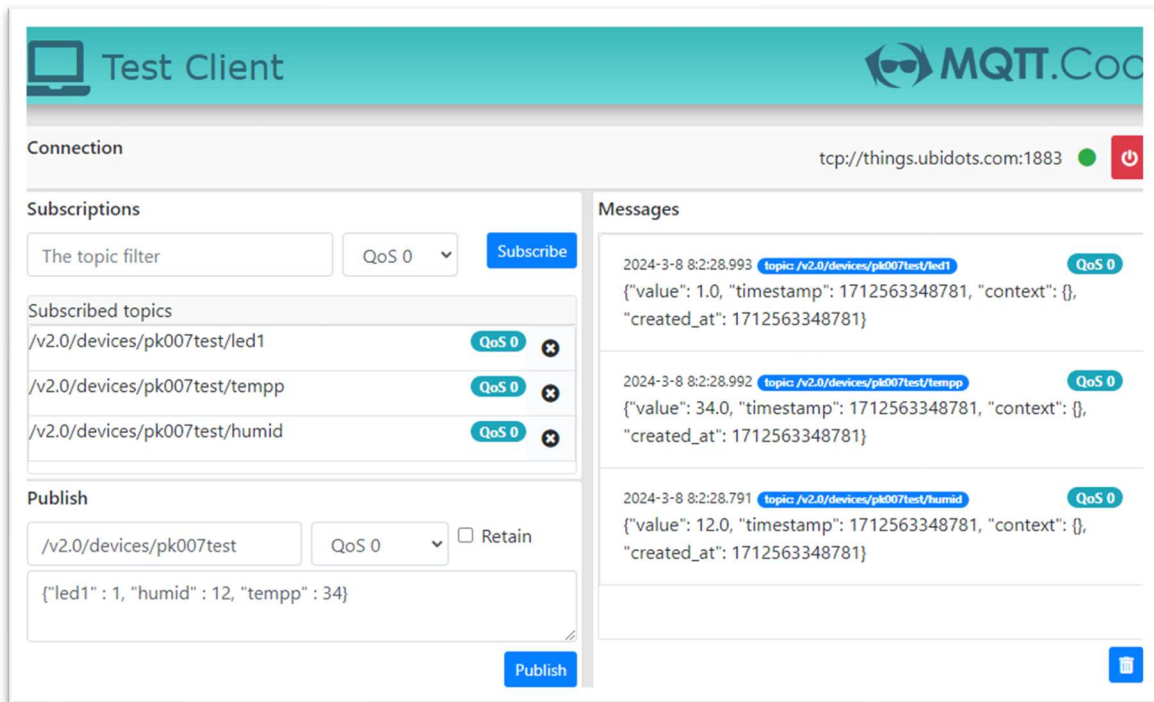
a few seconds ago

☐	Value	Name	Last updated ↓	
☐	67	humid	a minute ago	...
☐	13	tempp	a minute ago	...

Exp3_3/5: MQTT Client to/from Ubidots

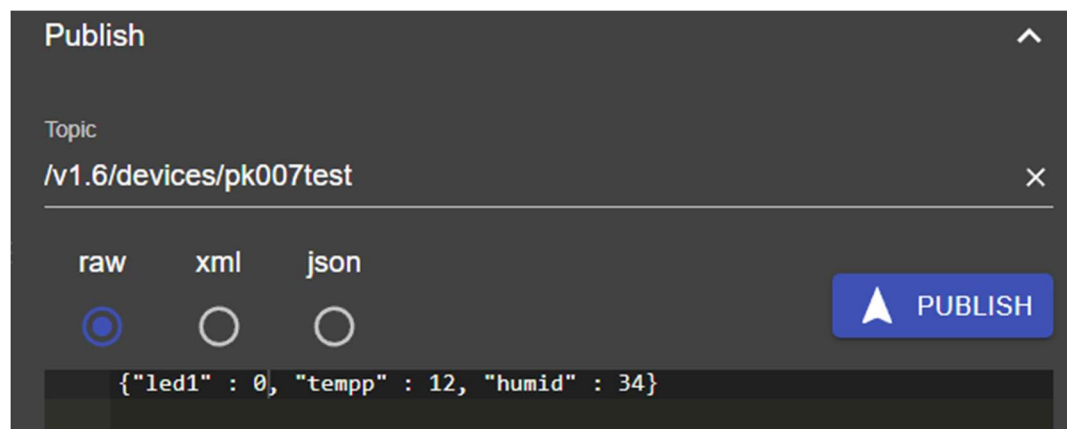
10. Code for Send Random Data to Ubidots and LED Control from Ubidots

10.1 Subscribe led1 variable and publish data

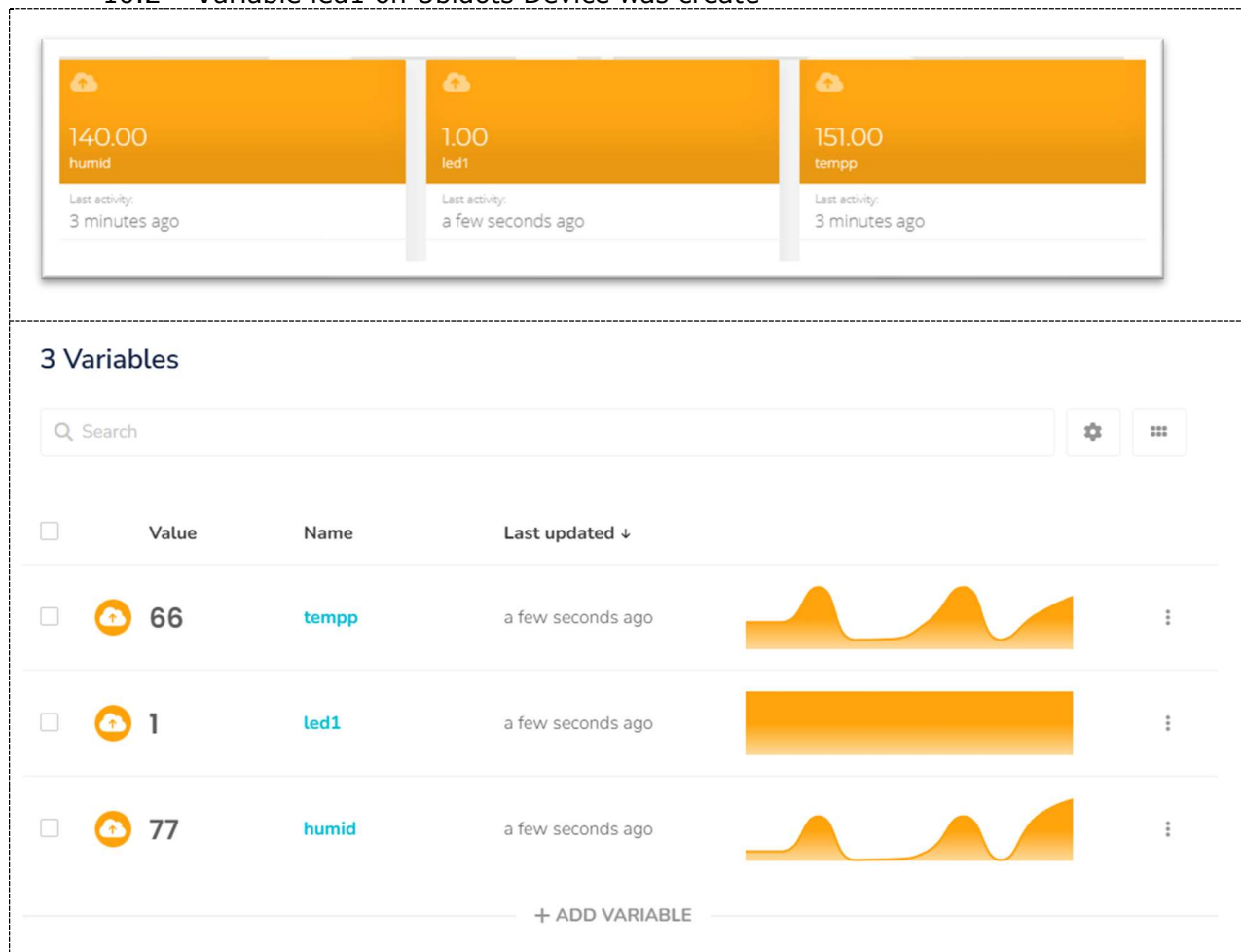


/v1.6/devices/pk007test/led1

{"led1" : 1, "humid" : 12, "temp" : 34}



10.2 Variable led1 on Ubidots Device was create



10.3 Control On → {"led1":1} for On, {"led1":0} for off

Off → Publish Topic `/v1.6/devices/pk007test/led1`, Data = {"led1":0}

The screenshot shows the MQTT Publish interface. The topic is `/v1.6/devices/pk007test/led1`. The format is set to `raw`. The data field contains `{"value" : 0}`. The PUBLISH button is visible.

Publish

Topic

`/v1.6/devices/pk007test/led1`

raw xml json

☒ ☐ ☐

PUBLISH

`{"value" : 0}`

On → Publish Topic `/v1.6/devices/pk007test/led1`, Data = {"led1":1}

The screenshot shows the MQTT Publish interface. The topic is `/v1.6/devices/pk007test/led1`. The format is set to `raw`. The data field contains `{"value" : 1}`. The PUBLISH button is visible.

Publish

Topic

`/v1.6/devices/pk007test/led1`

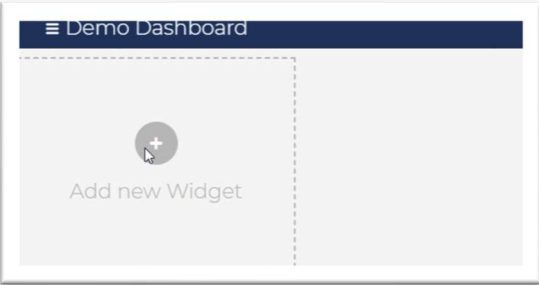
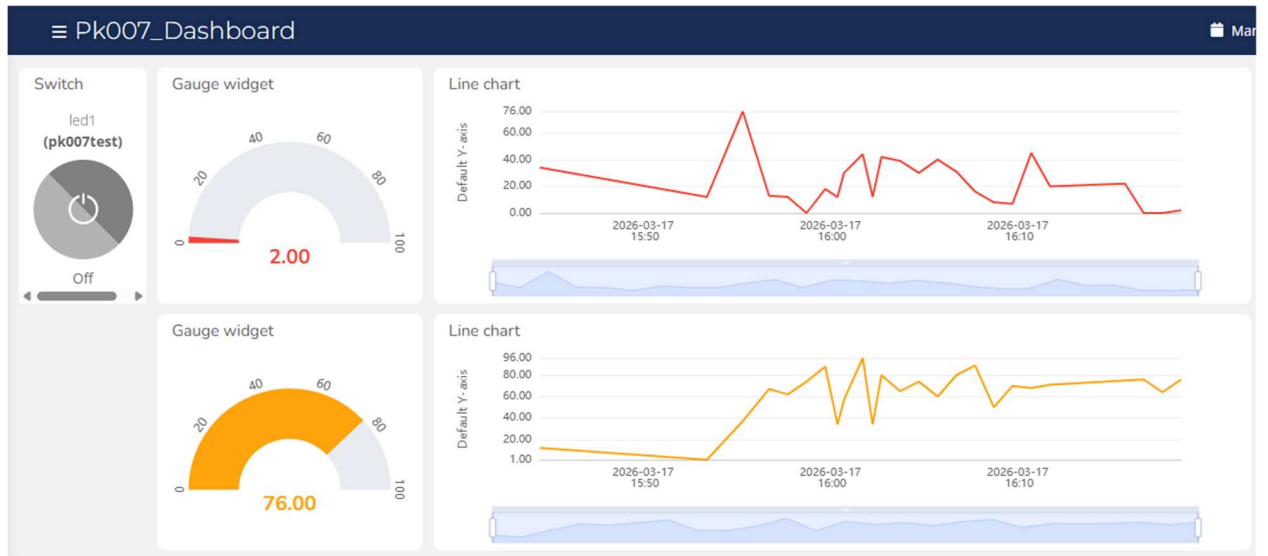
raw xml json

☒ ☐ ☐

PUBLISH

`{"value" : 1}`

11. Create Dashboard

	Data → Dashboard Add new Widget
Gauge Line chart	Gauge, Line chart Add Value → humid
Gauge Line chart	Gauge, Line chart Add Value → temp
Switch	Switch Add Value → led1
	

Control Switch On/Off

The MQTT Explorer interface displays a tree view of topics on the left and a history panel on the right. The tree view shows the following structure:

- industrial.api.ubidots.com
 - v1.6
 - devices
 - pk007test
 - tempp = {"value": 43.0, "timestamp": 1773739178070, "context": {}}
 - humid = {"value": 74.0, "timestamp": 1773739178070, "context": {}}
 - led1 = {"value": 0.0, "timestamp": 1773739122773, "context": {}, "value": 1}

The history panel shows the following messages:

- /v1.6/devices/pk007test/led1
 - {"value" : 0}
 - /v1.6/devices/pk007test/led1
 - {"value" : 1}

Publish

Topic

`/v1.6/devices/pk007test/led1`

raw

xml

json

`{"value" : 1}`

Publish

Topic

`/v1.6/devices/pk007test/led1`

raw

xml

json

`{"value" : 0}`

Exp3_4/5: ESP32 to/from Ubidots

12. Test Code for Send Random Data to Ubidots and Control LED D2 from Ubidots

```

#include <WiFi.h>
#include <PubSubClient.h>

const char *My_SSID = "IoT-Test";
const char *My_Pass = "pk12345678";

const char *MQTT_Server = "industrial.api.ubidots.com";
const char *MQTT_User = "BBUS-Vpmy8LzjBFQQireUISQIojhRS2vdNF";
const char *MQTT_Pass = "BBUS-Vpmy8LzjBFQQireUISQIojhRS2vdNF";
const char *PTopic1 = "/v1.6/devices/pk007test";
const char *STopic1 = "/v1.6/devices/pk007test/humid";
const char *STopic2 = "/v1.6/devices/pk007test/tempp";
const char *STopic3 = "/v1.6/devices/pk007test/led1";

#define MQTT_Port 1883
#define Test_LED1 2

WiFiClient espClient;
PubSubClient client(espClient);
long lastMsg = 0;
char msg[50];

void Setup_Wifi() {
  delay(10);
  Serial.println();
  Serial.print("Connecting to ");
  Serial.println(My_SSID);
  WiFi.begin(My_SSID, My_Pass);
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }

  randomSeed(micros());
  Serial.println("");
  Serial.println("WiFi connected");
  Serial.println("IP address: ");
  Serial.println(WiFi.localIP());
}

void reconnect() {
  while (!client.connected()) // Loop until we're reconnected
  {
    Serial.print("Attempting MQTT connection...");
    String clientId = "ESP32 Client-";
    clientId += String(random(0xffff), HEX); // Create a random client ID
    if (client.connect(clientId.c_str(), MQTT_User, MQTT_Pass)) // Attempt to connect
    {
      Serial.println("connected"); // Once connected, publish an announcement...
      client.subscribe(STopic1);
      client.subscribe(STopic2);
      client.subscribe(STopic3);
    } else {
      Serial.print("failed, rc=");
      Serial.print(client.state());
      Serial.println(" try again in 5 seconds");
      delay(5000);
    }
  }
}

void callback(char *topic, byte *payload, unsigned int length) {
  Serial.print("Message arrived [");
  Serial.print(topic);
  Serial.print("] ");
  for (int i = 0; i < length; i++) {
    Serial.print((char)payload[i]);
  }
}

```

```

if (topic[24] == STopic3[24]) {
  Serial.print(" -LED1->> ");
  Serial.print((char)payload[10]);
  if (payload[10] == '1')
    digitalWrite(Test_LED1, HIGH);
  else
    digitalWrite(Test_LED1, LOW);
}
Serial.println();
}

void setup() {
  pinMode(Test_LED1, OUTPUT);
  Serial.begin(115200);
  Setup_Wifi();
  client.setServer(MQTT_Server, MQTT_Port);
  client.setCallback(callback);
}

void loop() {
  if (!client.connected()) reconnect();
  client.loop();
  long now = millis();
  if (now - lastMsg > 60000) {
    lastMsg = now;
    int valueT = random(50);
    int valueH = random(50) + 50;
    snprintf(msg, 75, "{ \"tempp\" : %ld, \"humid\" : %ld }", valueT, valueH);
    Serial.print("Publish message: ");
    Serial.println(msg);
    client.publish(PTopic1, msg);
  }
}

```

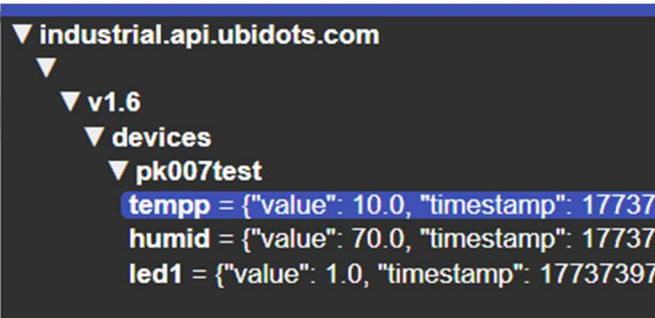
12.1 LED Control from Ubidots Dashboard >> On → Onboard LED D2 = On

Switch

led1
(pk007test)



On



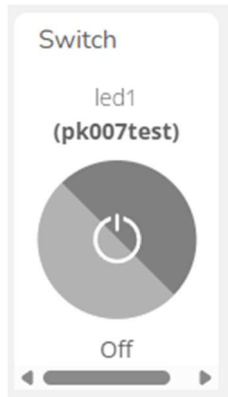
94

Output Serial Monitor X

Message (Enter to send message to 'DOIT ESP32 DEVKIT V1' on 'COM12')

16:30:11.227 -> Message arrived [/v1.6/devices/pk007test/led1] {"value": 1.0, "

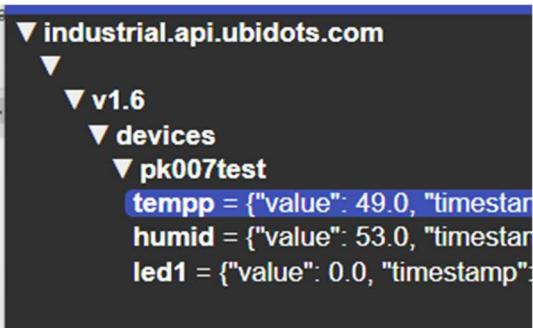
12.2 LED Control from Ubidots Dashboard >> Off → Onboard LED D2 = Off



Switch

led1
(pk007test)

Off



```
▼ industrial.api.ubidots.com
  ▼ v1.6
    ▼ devices
      ▼ pk007test
        temp = {"value": 49.0, "timestamp": 163056074}
        humid = {"value": 53.0, "timestamp": 163056074}
        led1 = {"value": 0.0, "timestamp": 163056074}
```

Output Serial Monitor X

Message (Enter to send message to 'DOIT ESP32 DEVKIT V1' on 'COM12')

```
16:30:11.227 -> Message arrived [/v1.6/devices/pk007test/led1] {"value": 1.0, "timestamp": 163011227}
16:30:55.354 -> Publish message: { "temp" : 49, "humid": 53 }
16:30:56.074 -> Message arrived [/v1.6/devices/pk007test/temp] {"value": 49.0, "timestamp": 163056074}
16:30:56.119 -> Message arrived [/v1.6/devices/pk007test/humid] {"value": 53.0, "timestamp": 163056119}
16:31:30.607 -> Message arrived [/v1.6/devices/pk007test/led1] {"value": 0.0, "timestamp": 163030607}
16:31:55.374 -> Publish message: { "temp" : 27, "humid": 93 }
16:31:56.090 -> Message arrived [/v1.6/devices/pk007test/humid] {"value": 93.0, "timestamp": 163056090}
16:31:56.123 -> Message arrived [/v1.6/devices/pk007test/temp] {"value": 27.0, "timestamp": 163056123}
```


Exp3-5/5: DHT22 on ESP32 to Ubidots and Control 2LED on D22 D23

13. Coding for Test

```

#include <WiFi.h>
#include <PubSubClient.h>
#include "DHTesp.h"

const char *My_SSID = "IoT-Test";
const char *My_Pass = "pk12345678";

const char *MQTT_Server = "industrial.api.ubidots.com";
const char *MQTT_User = "BBUS-Vpmy8LzjBFQQireUISQIojhRS2vdNF";
const char *MQTT_Pass = "BBUS-Vpmy8LzjBFQQireUISQIojhRS2vdNF";
const char *PTopic1 = "/v1.6/devices/pk007test";
const char *STopic1 = "/v1.6/devices/pk007test/humid";
const char *STopic2 = "/v1.6/devices/pk007test/tempp";
const char *STopic3 = "/v1.6/devices/pk007test/led1";
const char *STopic4 = "/v1.6/devices/pk007test/led2";

#define MQTT_Port 1883
#define Test_LED1 22
#define Test_LED2 23
#define Pin_DHT22 4

DHTesp dht;
WiFiClient espClient;
PubSubClient client(espClient);
long lastMsg = 0;
char msg[50];

void Setup_Wifi() {
  delay(10);
  Serial.println();
  Serial.print("Connecting to ");
  Serial.println(My_SSID);
  WiFi.begin(My_SSID, My_Pass);
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }
  randomSeed(micros());
  Serial.println("");
  Serial.println("WiFi connected");
  Serial.println("IP address: ");
  Serial.println(WiFi.localIP());
}

void reconnect() {
  while (!client.connected()) // Loop until we're reconnected
  {
    Serial.print("Attempting MQTT connection...");
    String clientId = "ESP32 Client-";
    clientId += String(random(0xffff), HEX); // Create a random client ID
    if (client.connect(clientId.c_str(), MQTT_User, MQTT_Pass)) // Attempt to connect
    {
      Serial.println("connected"); // Once connected, publish an announcement...
      client.subscribe(STopic1);
      client.subscribe(STopic2);
      client.subscribe(STopic3);
      client.subscribe(STopic4);
    } else {
      Serial.print("failed, rc=");
      Serial.print(client.state());
      Serial.println(" try again in 5 seconds");
      delay(5000);
    }
  }
}

```

```

void callback(char *topic, byte *payload, unsigned int length) {
  Serial.print("Message arrived [");
  Serial.print(topic);
  Serial.print("] ");
  for (int i = 0; i < length; i++) Serial.print((char)payload[i]);

  if (strstr(topic, "led1")) {
    Serial.print(" >> Yes LED1 = ");
    Serial.print((char)payload[10]);
    digitalWrite(Test_LED1, (payload[10] == '1' ? HIGH : LOW));
  }

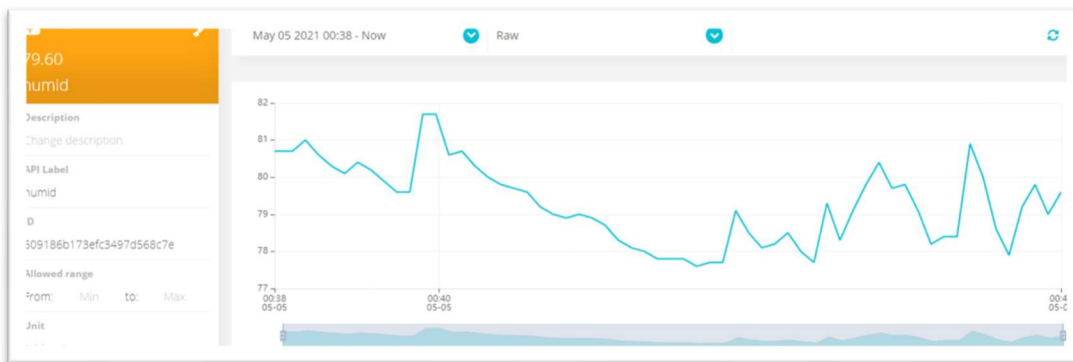
  if (strstr(topic, "led2")) {
    Serial.print(" >> Yes LED2 = ");
    Serial.print((char)payload[10]);
    digitalWrite(Test_LED2, (payload[10] == '1' ? HIGH : LOW));
  }

  Serial.println();
}

void setup() {
  pinMode(Test_LED1, OUTPUT);
  pinMode(Test_LED2, OUTPUT);
  dht.setup(Pin_DHT22, DHTesp::DHT22);
  Serial.begin(115200);
  Setup_Wifi();
  client.setServer(MQTT_Server, MQTT_Port);
  client.setCallback(callback);
}

void loop() {
  if (!client.connected()) reconnect();
  client.loop();
  long now = millis();
  if (now - lastMsg > 60000) {
    lastMsg = now;
    float humidity = dht.getHumidity();
    float temperature = dht.getTemperature();
    snprintf(msg, 75, "{ \"humid\" : %.2f, \"tempp\" : %.2f }", humidity, temperature);
    Serial.print("Publish message: ");
    Serial.println(msg);
    client.publish(PTopic1, msg);
  }
}

```



Short Time



Long Time

14. Update Dashboard and Share

Switch

led1
(pk007test)



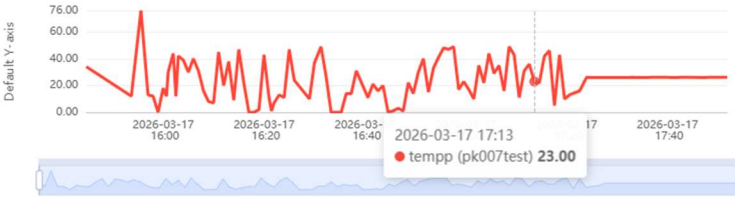
On

Gauge widget



26.10

Line chart



temp (pk007test) 23.00

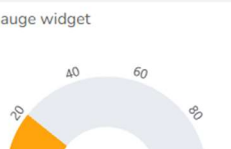
Switch

led2
(pk007test)



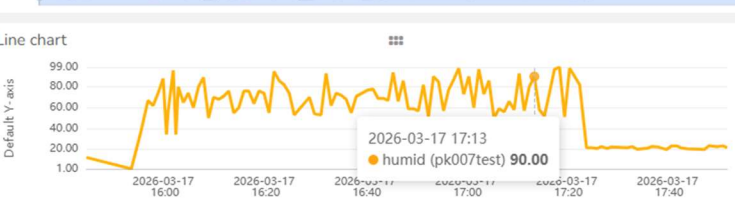
On

Gauge widget



21.50

Line chart



humid (pk007test) 90.00


myDashboard

Click my Dashboard

Dashboards

Search

Name

Organization

Created

Pk007_Dashboard

-

2026-03-17 16:00

Share

Duplicate

Lock

Assign organization

Pin to top

Delete

Click Share
Create Link

Pk007_Dashboard

×

Share dashboard

Language

Set by account

▼

Date range

Dashboard's default

▼

Show navbar

☐

Show context bar

☐

Public link

English (United States)

Embed

`<iframe width="100%" height="100"`



URL

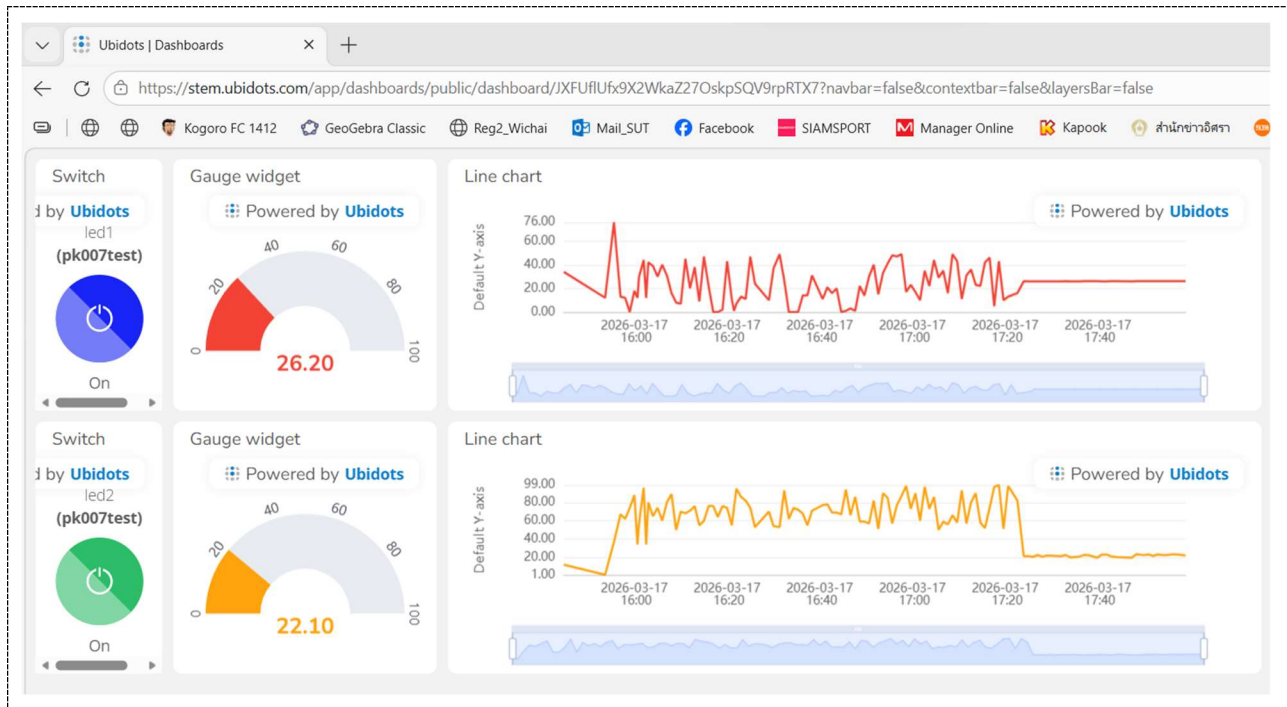
`https://stem.ubidots.com/app/dashbo`



+ CREATE NEW LINK

DONE

15. Test with Browser

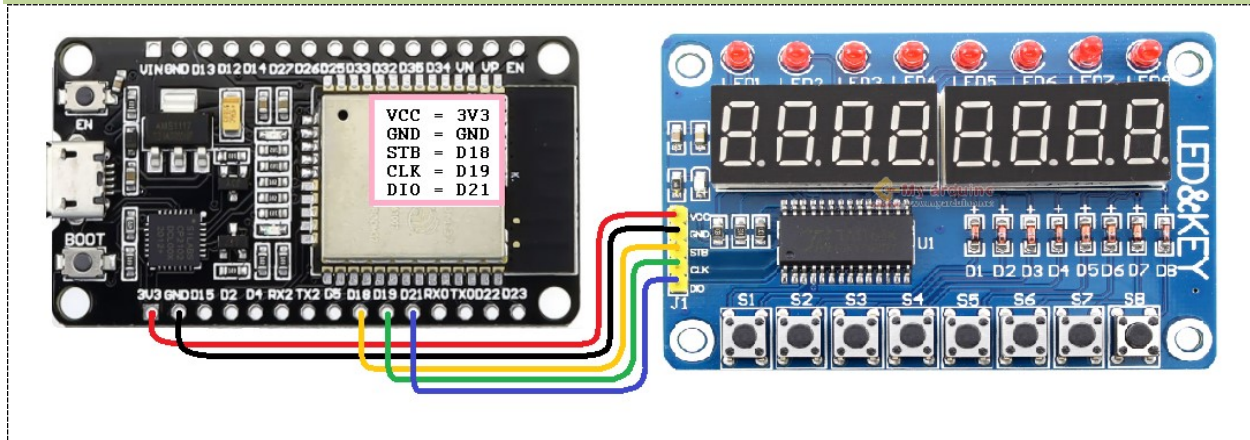


Question

Q201: ทดสอบบอร์ดเกอร์ Ubidots Platform ให้ควบคุม 4 LED, แสดงค่า 4 จำนวน DHT22 {Temperature, Humidity} + Random ระหว่าง 0.00 ถึง 49.99 +Random ระหว่าง 50.00 ถึง 99.99 โดยเขียนแสดงวิธีขั้นตอนการทำให้ละเอียดสุด พร้อมแนบลิงค์การแชร์ Dashboard ของตัวเอง

3/3: Oil Price Display via MQTT (Update 20260315)

การทดลองที่ 4/4: Application with Publish and Subscribe



Q301: ออกแบบป้ายราคาน้ำมันหน้าปั้ม

- กำหนดให้มีน้ำมัน 8 ชนิด ราคาจะถูกกำหนดไว้ที่ แต่ละ Topic บน MQTT Server
- เมื่อกดสวิตช์ S1 ไฟ LED1 จะติดค้างไว้ และแสดงราคาน้ำมันชนิดที่ 1
- เมื่อกดสวิตช์ S2 ไฟ LED2 จะติดค้างไว้ และแสดงราคาน้ำมันชนิดที่ 2
-
- เมื่อกดสวิตช์ S8 ไฟ LED8 จะติดค้างไว้ และแสดงราคาน้ำมันชนิดที่ 8
- ขณะที่แสดงราคาน้ำมัน หากราคามีการเปลี่ยนแปลง ค่าที่แสดงก็จะเปลี่ยนตามด้วย

ข้อกำหนดการทดสอบ

- นักศึกษาสามารถแก้ไขราคาได้ โดยการ Publish ราคาระหว่าง 0.00 – 999999.99 บาท/ลิตร
- กำหนดราคาเก็บไว้ที่
- กำหนดห้องที่ใช้ทดสอบ ดังนี้
 - o Name Room.1 Topic = SUT_Micro.2021_R01qaz
 - o Name Room.2 Topic = SUT_Micro.2021_R02wsx
 - o Name Room.3 Topic = SUT_Micro.2021_R03edc
 - o Name Room.4 Topic = SUT_Micro.2021_R04rfv
 - o Name Room.5 Topic = SUT_Micro.2021_R05tgb
 - o Name Room.6 Topic = SUT_Micro.2021_R06yhn
 - o Name Room.7 Topic = SUT_Micro.2021_R07ujm
 - o Name Room.8 Topic = SUT_Micro.2021_R08ikl

Arduino ESP32 Library

1. TM1638plus by Gavin Lyons Version 2.2.0

TM1638plus by Gavin Lyons <glyons66@hotmail.com>

TM1638plus is an Arduino library to control TM1638 & TM1637 seven segment modules. It supports push buttons, LEDs and the decimal point LED features of these modules. Can display ASCII, Decimal, Hexadecimal and text strings. Small Memory footprint.

[More info](#)

2.2.0



INSTALL

2. PubSubClient3 by Nick O'Leary Version 3.3.0

PubSubClient3 by Nick O'Leary <nick.oleary@gmail.com>

A client library for MQTT messaging. MQTT is a lightweight messaging protocol ideal for small devices. This library allows you to send and receive MQTT messages. It supports the latest MQTT 3.1.1 protocol and can be configured to use the older MQTT 3.1 if needed. It supports all Arduino Ethernet Client compatible hardware, including the Intel...

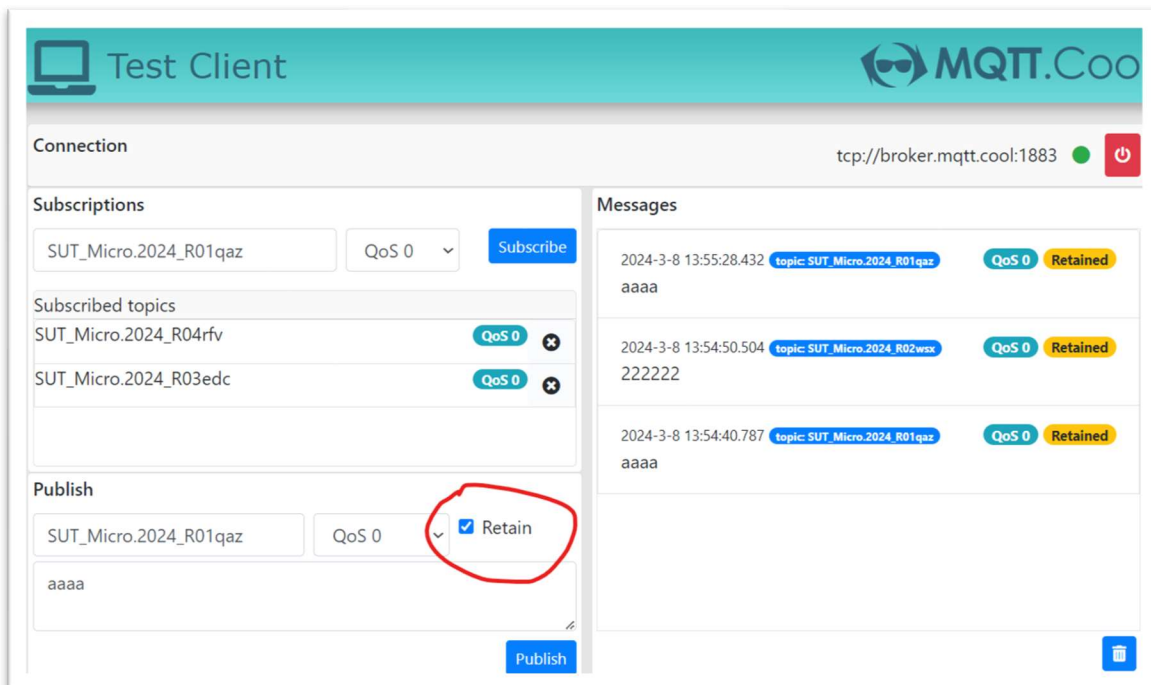
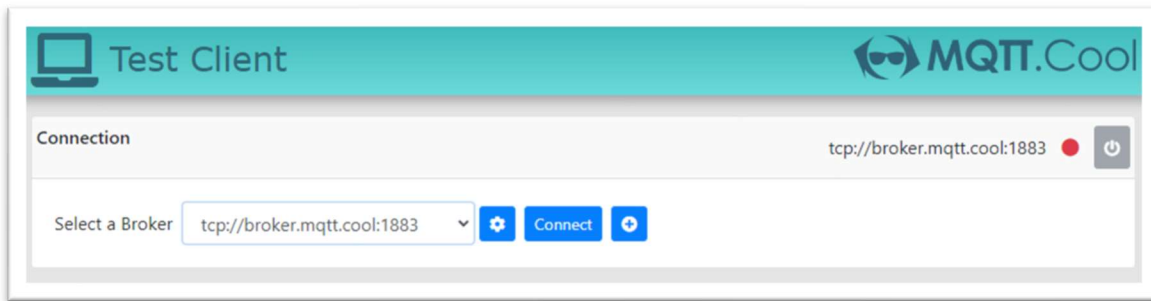
[More info](#)

3.3.0



INSTALL

<https://testclient-cloud.mqtt.cool/>



Subscribe / Publish Topic

- SUT_Micro.2024_R01qaz
- SUT_Micro.2024_R02wsx
- SUT_Micro.2024_R03edc
- SUT_Micro.2024_R04rfv
- SUT_Micro.2024_R05tgb
- SUT_Micro.2024_R06yhn
- SUT_Micro.2024_R07ujm
- SUT_Micro.2024_R08ikl

TestQ301_1of5 – LED Blink

```
#include <TM1638plus.h>

#define Brd_STB 18 // strobe = GPIO connected to strobe line of module
#define Brd_CLK 19 // clock = GPIO connected to clock line of module
#define Brd_DIO 21 // data = GPIO connected to data line of module
bool high_freq = true; // default false,, If using a high freq CPU > ~100 MHZ set to true.

TM1638plus tm(Brd_STB, Brd_CLK, Brd_DIO, high_freq);
int xDelay = 1000;

void All2BlinkLED(void) {
    for (int i = 0; i < 2; i++) {
        tm.setLEDs(0xff00); // All On
        tm.displayText("On ff00");
        delay(xDelay);
        tm.setLEDs(0x0000); // All Off
        tm.displayText("Off 0000");
        delay(xDelay);
    }
}

void setup() {
    tm.displayBegin();
}

void loop() {
    char workStr[11];
    All2BlinkLED();
    for (int i = 1; i <= 8; i++) {
        int positionLED = 1 << (i - 1); // 0x00
        tm.setLEDs(positionLED);
        sprintf(workStr, "C=%01d %04x", i, positionLED);
        tm.displayText(workStr);
        delay(xDelay);
    }
}
```

TestQ301_2of5 – Switch Select LED

```
#include <TM1638plus.h>

#define Brd_STB 18 // strobe = GPIO connected to strobe line of module
#define Brd_CLK 19 // clock = GPIO connected to clock line of module
#define Brd_DIO 21 // data = GPIO connected to data line of module
bool high_freq = true; //default false,, If using a high freq CPU > ~100 MHZ set to true.

TM1638plus tm(Brd_STB, Brd_CLK , Brd_DIO, high_freq);
int xDelay = 200;

void All2BlinkLED(void) {
  for (int i = 0; i < 2; i++) {
    tm.setLEDs(0xff00); // All On
    tm.displayText("On ff00");
    delay(xDelay * 5);
    tm.setLEDs(0x0000); // All Off
    tm.displayText("Off 0000");
    delay(xDelay * 5);
  }
}

void setup() {
  Serial.begin(115200);
  tm.displayBegin();
  All2BlinkLED();
}

void loop() {
  char workStr[11];
  while (1) {
    int inputSelect;
    uint8_t buttons = tm.readButtons();
    Serial.println(buttons, HEX);
    if (buttons >> 0 & 1) inputSelect = 1;
    else if (buttons >> 1 & 1) inputSelect = 2;
    else if (buttons >> 2 & 1) inputSelect = 3;
    else if (buttons >> 3 & 1) inputSelect = 4;
    else if (buttons >> 4 & 1) inputSelect = 5;
    else if (buttons >> 5 & 1) inputSelect = 6;
    else if (buttons >> 6 & 1) inputSelect = 7;
    else if (buttons >> 7 & 1) inputSelect = 8;
    else inputSelect = 0;
    int positionLED = 1 << (inputSelect + 7); // xx00
    tm.setLEDs(positionLED);
    sprintf(workStr, "C=%01d %04x", inputSelect, positionLED);
    tm.displayText(workStr);
    delay(xDelay);
  }
}
```

TestQ301_3of5 – Switch Select LED with Subroutine

```
#include <TM1638plus.h>

#define Brd_STB 18 // strobe = GPIO connected to strobe line of module
#define Brd_CLK 19 // clock = GPIO connected to clock line of module
#define Brd_DIO 21 // data = GPIO connected to data line of module
bool high_freq = true; //default false,, If using a high freq CPU > ~100 MHZ set to true.

TM1638plus tm(Brd_STB, Brd_CLK, Brd_DIO, high_freq);
int xDelay = 200;

void All2BlinkLED(void) {
  for (int i = 0; i < 2; i++) {
    tm.setLEDs(0xff00); // All On
    tm.displayText("On ff00");
    delay(xDelay * 5);
    tm.setLEDs(0x0000); // All Off
    tm.displayText("Off 0000");
    delay(xDelay * 5);
  }
}

int Scan_CaseRun(void) {
  int inputSelect = 0;
  uint8_t buttons = tm.readButtons();
  if (buttons >> 0 & 1) inputSelect = 1;
  else if (buttons >> 1 & 1) inputSelect = 2;
  else if (buttons >> 2 & 1) inputSelect = 3;
  else if (buttons >> 3 & 1) inputSelect = 4;
  else if (buttons >> 4 & 1) inputSelect = 5;
  else if (buttons >> 5 & 1) inputSelect = 6;
  else if (buttons >> 6 & 1) inputSelect = 7;
  else if (buttons >> 7 & 1) inputSelect = 8;
  else inputSelect = 0;
  return inputSelect;
}

void Display_CaseRun(int inputSelect) {
  char workStr[11];
  int positionLED = 1 << (inputSelect + 7); // xx00
  tm.setLEDs(positionLED);
  sprintf(workStr, "C=%01d %04x", inputSelect, positionLED);
  tm.displayText(workStr);
  delay(xDelay);
}

void setup() {
  Serial.begin(115200);
  tm.displayBegin();
  All2BlinkLED();
}

void loop() {
  char workStr[11];
  int positionLED, inputSelect, scanKeyInput;
  while (1) {
    delay(250);
    scanKeyInput = Scan_CaseRun();
    if (scanKeyInput != 0) {
      inputSelect = scanKeyInput;
      Display_CaseRun(inputSelect);
    }
  }
}
```

TestQ301_4of5 – Read Oil Price

```
#include <WiFi.h>
#include <PubSubClient.h>

const char* ssid = "Test1234";
const char* password = "0816601929";

const char *mqtt_server = "broker.mqtt.cool";

const char *topic1 = "SUT_Micro.2024_R01qaz";

#define MSG_BUFFER_SIZE (50)
#define testLED 2

WiFiClient espClient;
PubSubClient client(espClient);
int sts_LED = 1;
unsigned long timeNow, timeLast;
char msg[MSG_BUFFER_SIZE];

void setup_wifi() {
  delay(10);
  Serial.println();          Serial.print("Connecting to ");
  Serial.println(ssid);      WiFi.mode(WIFI_STA);
  WiFi.begin(ssid, password);
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }
  randomSeed(micros());
  Serial.println("");
  Serial.println("WiFi connected");
  Serial.println("IP address: ");
  Serial.println(WiFi.localIP());
}

void callback(char* topic, byte* payload, unsigned int length) {
  Serial.print("Message arrived [");
  Serial.print(topic);
  Serial.print("] ");
  for (int i = 0; i < length; i++) {
    Serial.print((char)payload[i]);
  }
  Serial.println();
}

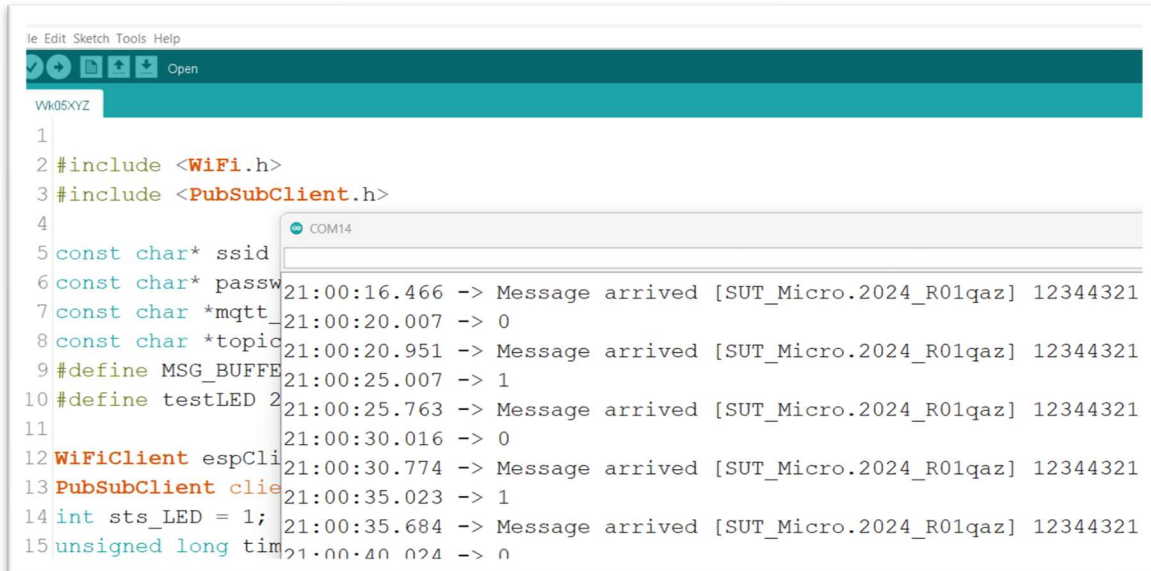
void reconnect() {
  while (!client.connected()) {
    Serial.print("Attempting MQTT connection...");
    String clientId = "ESP8266Client-";
    clientId += String(random(0xffff), HEX);
    if (client.connect(clientId.c_str())) {
      Serial.println("connected");
      client.subscribe(topic1);
    } else {
      Serial.print("failed, rc=");
      Serial.print(client.state());
      Serial.println(" try again in 5 seconds");
      delay(5000);
    }
  }
}

void setup() {
  pinMode(testLED, OUTPUT);
  Serial.begin(115200);
  setup_wifi();
  client.setServer(mqtt_server, 1883);
  client.setCallback(callback);
  timeLast = millis();
}
```

```

void loop() {
  timeNow = millis();
  if (!client.connected()) reconnect();
  client.loop();
  if ((timeNow - timeLast) > 5000) {
    timeLast = timeNow;
    digitalWrite(testLED, sts_LED);
    sts_LED = 1 - sts_LED;
    client.subscribe(topic1);
    Serial.println(sts_LED);
  }
}

```



The screenshot shows the Arduino IDE interface. The sketch editor on the left contains the following code:

```

1
2 #include <WiFi.h>
3 #include <PubSubClient.h>
4
5 const char* ssid = "VW05XYZ";
6 const char* password = "12345678";
7 const char* mqtt_server = "broker.mqtt.cool";
8 const char* topic1 = "SUT_Micro.2024_R01qaz";
9 #define MSG_BUFFER_SIZE 1024
10 #define testLED 2
11
12 WiFiClient espClient;
13 PubSubClient client(espClient, mqtt_server, WiFiClient);
14 int sts_LED = 1;
15 unsigned long timeLast = 0;

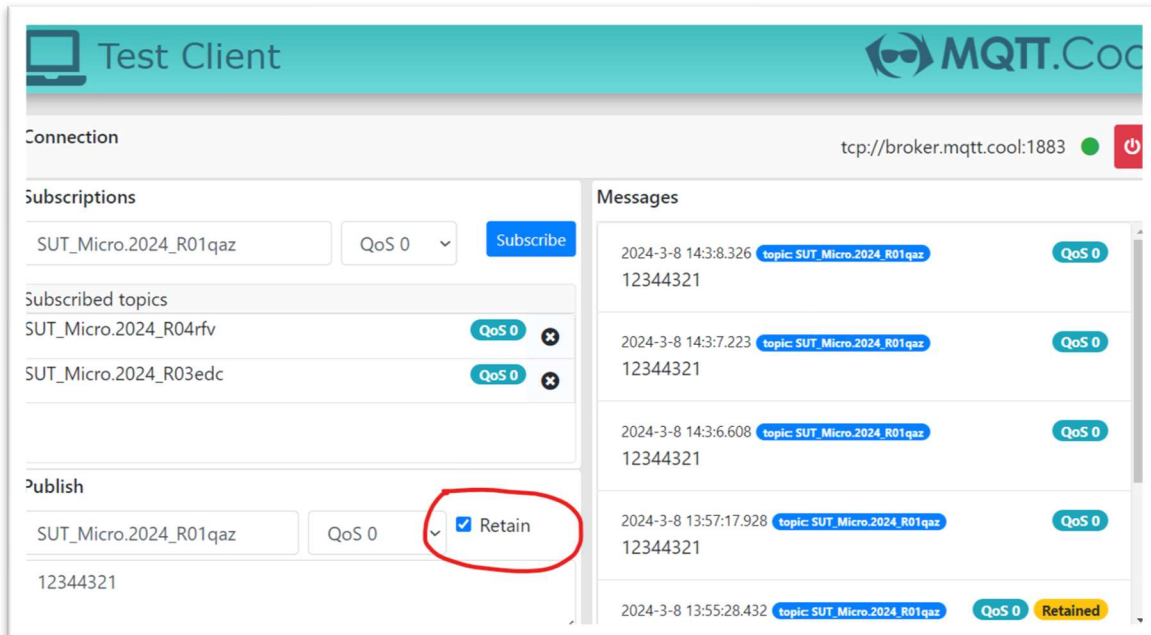
```

The serial monitor on the right shows the output of the program, displaying timestamps and messages received from the topic SUT_Micro.2024_R01qaz:

```

21:00:16.466 -> Message arrived [SUT_Micro.2024_R01qaz] 12344321
21:00:20.007 -> 0
21:00:20.951 -> Message arrived [SUT_Micro.2024_R01qaz] 12344321
21:00:25.007 -> 1
21:00:25.763 -> Message arrived [SUT_Micro.2024_R01qaz] 12344321
21:00:30.016 -> 0
21:00:30.774 -> Message arrived [SUT_Micro.2024_R01qaz] 12344321
21:00:35.023 -> 1
21:00:35.684 -> Message arrived [SUT_Micro.2024_R01qaz] 12344321
21:00:40.024 -> 0

```



The screenshot shows the MQTT Client web interface. The connection is established to tcp://broker.mqtt.cool:1883. The interface is divided into three main sections:

- Subscriptions:** A list of subscribed topics: SUT_Micro.2024_R04rfv, SUT_Micro.2024_R03edc, and SUT_Micro.2024_R01qaz. Each topic has a QoS 0 button and a delete icon.
- Publish:** A form to publish messages to the topic SUT_Micro.2024_R01qaz. The QoS is set to 0, and the **Retain** checkbox is checked (highlighted with a red circle).
- Messages:** A list of received messages, each with a timestamp, topic, and QoS 0. The messages are:
 - 2024-3-8 14:3:8.326 topic: SUT_Micro.2024_R01qaz 12344321
 - 2024-3-8 14:3:7.223 topic: SUT_Micro.2024_R01qaz 12344321
 - 2024-3-8 14:3:6.608 topic: SUT_Micro.2024_R01qaz 12344321
 - 2024-3-8 13:57:17.928 topic: SUT_Micro.2024_R01qaz 12344321
 - 2024-3-8 13:55:28.432 topic: SUT_Micro.2024_R01qaz 12344321 (Retained)

TestQ301_5of5 – Oil Price (Ok)

```
#include <WiFi.h>
#include <PubSubClient.h>
#include <TM1638plus.h>

#define Brd_STB 18
#define Brd_CLK 19
#define Brd_DIO 21
bool high_freq = true;

const char* ssid = "SUT_IoTs";
const char* password = "MaiMeeJingJing";

const char *mqtt_server = "broker.mqtt.cool";

const char *topic1 = "SUT_Micro.2024_R01qaz";
const char *topic2 = "SUT_Micro.2024_R02wsx";
const char *topic3 = "SUT_Micro.2024_R03edc";
const char *topic4 = "SUT_Micro.2024_R04rfv";
const char *topic5 = "SUT_Micro.2024_R05tgb";
const char *topic6 = "SUT_Micro.2024_R06yhn";
const char *topic7 = "SUT_Micro.2024_R07ujm";
const char *topic8 = "SUT_Micro.2024_R08ikl";

TM1638plus tm(Brd_STB, Brd_CLK, Brd_DIO, high_freq);
WiFiClient espClient;
PubSubClient client(espClient);

long lastMsg = 0;
int TestLED = 2;
int inputSelect = 1, sts_TestLED = 1;
int xDelay = 200;
char workStr[11];
char msg[50];

void All2BlinkLED(void) {
  for (int i = 0; i < 2; i++) {
    tm.setLEDs(0xff00); // All On
    tm.displayText("On ff00");
    delay(xDelay * 5);
    tm.setLEDs(0x0000); // All Off
    tm.displayText("Off 0000");
    delay(xDelay * 5);
  }
}

int Scan_CaseRun(void) {
  int inputSelect = 0;
  uint8_t buttons = tm.readButtons();
  if (buttons >> 0 & 1) inputSelect = 1;
  else if (buttons >> 1 & 1) inputSelect = 2;
  else if (buttons >> 2 & 1) inputSelect = 3;
  else if (buttons >> 3 & 1) inputSelect = 4;
  else if (buttons >> 4 & 1) inputSelect = 5;
  else if (buttons >> 5 & 1) inputSelect = 6;
  else if (buttons >> 6 & 1) inputSelect = 7;
  else if (buttons >> 7 & 1) inputSelect = 8;
  else inputSelect = 0;
  return inputSelect;
}

void Display_CaseRun(int inputSelect) {
  int positionLED = 1 << (inputSelect + 7); // xx00
  tm.setLEDs(positionLED);
  sprintf(workStr, "C=%01d %04x", inputSelect, positionLED);
  tm.displayText(workStr);
  delay(xDelay);
}
```

```

void setup_wifi() {
  delay(10);
  Serial.println();
  Serial.print("Connecting to ");
  Serial.println(ssid);
  WiFi.begin(ssid, password);
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);    Serial.print(".");
  }
  randomSeed(micros());
  Serial.println("");
  Serial.println("WiFi connected");
  Serial.println("IP address: ");
  Serial.println(WiFi.localIP());
  pinMode(TestLED, OUTPUT);
}

void callback(char* topic, byte* payload, unsigned int length)
{ char myPayload[50];
  Serial.print("Message arrived >> ");
  for (int i = 0; i < length; i++)
  { Serial.print((char)payload[i]);
    myPayload[i] = payload[i];
    myPayload[i + 1] = '\0'; // End of String
  }
  Serial.print("\n ---> "); Serial.print(myPayload);
  for (int i = 0; i < sizeof(myPayload); i++)
  if ((myPayload[i] == '\0') && (i < 16)) {
    myPayload[i] = ' ';
    myPayload[i + 1] = '\0';
  }
  tm.displayText(myPayload);
}

void reconnect()
{ while (!client.connected())          // Loop until we're reconnected
  { Serial.print("Attempting MQTT connection...");
    String clientId = "Wichai_Client-";
    clientId += String(random(0xffff), HEX); // Create a random client ID
    if (client.connect(clientId.c_str())) // Attempt to connect
    { Serial.print("connected >> ");      // Once connected
      if (inputSelect == 1) strcpy(msg, topic1);
      if (inputSelect == 2) strcpy(msg, topic2);
      if (inputSelect == 3) strcpy(msg, topic3);
      if (inputSelect == 4) strcpy(msg, topic4);
      if (inputSelect == 5) strcpy(msg, topic5);
      if (inputSelect == 6) strcpy(msg, topic6);
      if (inputSelect == 7) strcpy(msg, topic7);
      if (inputSelect == 8) strcpy(msg, topic8);
      client.subscribe(msg);
      Serial.println(msg);
    } else
    { Serial.print("failed, rc=");
      Serial.print(client.state());
      Serial.println(" try again in 5 seconds");
      delay(5000);
    }
  }
}

//-----
void setup() {
  Serial.begin(115200);
  tm.displayBegin();
  All2BlinkLED();
  setup_wifi();
  client.setServer(mqtt_server, 1883);
  client.setCallback(callback);
  pinMode(TestLED, OUTPUT);
}

```



```
//-----
void loop() {
  int scanKeyInput;
  long now = millis();
  if (!client.connected()) reconnect();
  client.loop();
  if (now - lastMsg > 250)
  { lastMsg = now;
    sts_TestLED = 1 - sts_TestLED;
    digitalWrite(TestLED, sts_TestLED);
    scanKeyInput = Scan_CaseRun();
    if (scanKeyInput != 0) {
      inputSelect = scanKeyInput;
      Display_CaseRun(inputSelect);
      client.disconnect(); delay(1000);
      reconnect();
    }
  }
}
```

The screenshot displays the MQTT client interface. On the left, the 'Subscriptions' panel shows three subscribed topics: SUT_Micro.2024_R03edc, SUT_Micro.2024_R02wsx, and SUT_Micro.2024_R01qaz, each with a 'QoS 0' label and a close button. Below this, the 'Publish' section shows a message being sent to SUT_Micro.2024_R04rfv with 'QoS 0' and the 'Retain' checkbox checked. On the right, the 'Messages' panel shows a list of received messages with timestamps, topics, and values. The messages are: 44.56 (topic SUT_Micro.2024_R04rfv), 34.56 (topic SUT_Micro.2024_R03edc), 23.45 (topic SUT_Micro.2024_R02wsx), and 12.34 (topic SUT_Micro.2024_R01qaz).

The screenshot shows the Arduino IDE with the code from the previous block. The serial monitor is open, displaying the output of the program. The output shows the device connecting to the WiFi network 'Test1234' and then attempting an MQTT connection to the topic SUT_Micro.2024_R01qaz. The message 'Message arrived >> 12344321' is received.

```
7 #define Brd_CLK 19
8 #define Brd_DIO 21
9 bool high_freq = true;
10
11 const char* ssid
12 const char* pass
13
14 const char *mqtt
15
16 const char *topi
17 const char *topi
18 const char *topi
19 const char *topi
20 const char *topi
```

```
21:13:10.681 ->
21:13:10.681 -> Connecting to Test1234
21:13:11.295 -> .....
21:13:13.277 -> WiFi connected
21:13:13.277 -> IP address:
21:13:13.277 -> 192.168.1.4
21:13:13.277 -> Attempting MQTT connection...connected >> SUT_Micro.2024_R01qaz
21:13:14.978 -> Message arrived >> 12344321
21:13:14.978 -> ----> 12344321
```